Memory System

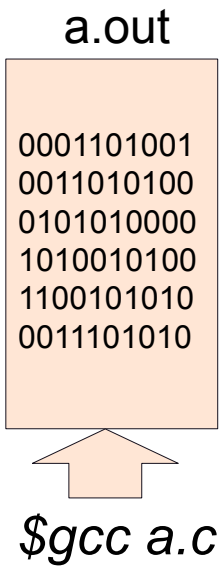
CS2600 (3003)

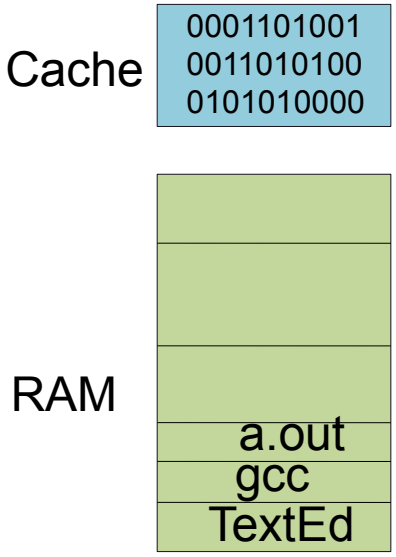
By

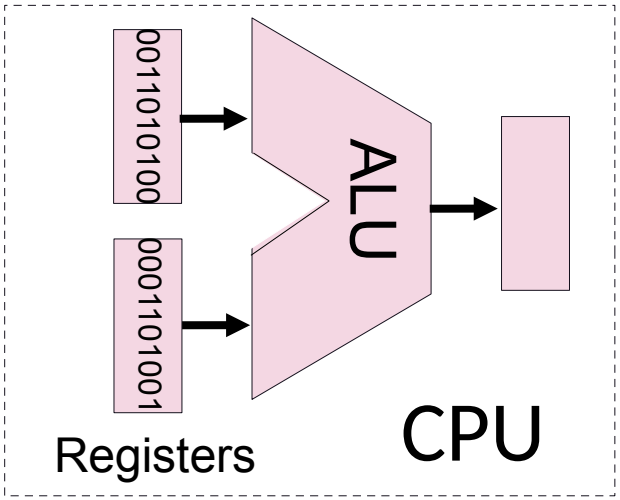
Dr. Sk Subidh Ali

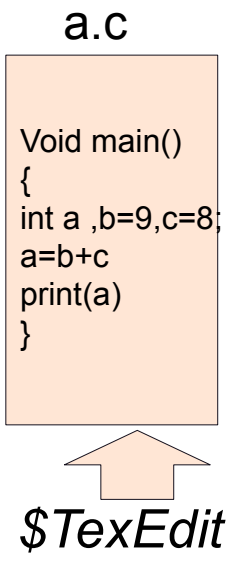
- Memory Hierarchy

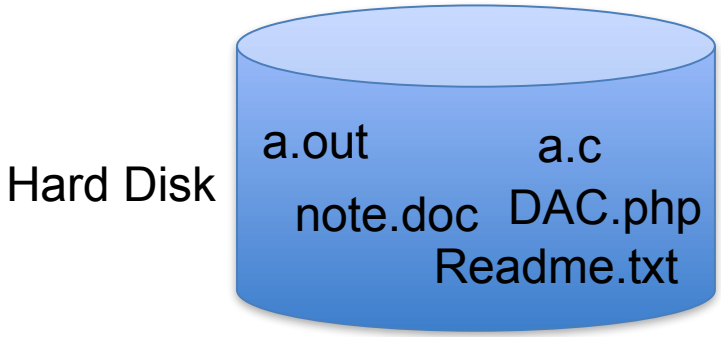
Memory Hierarchy

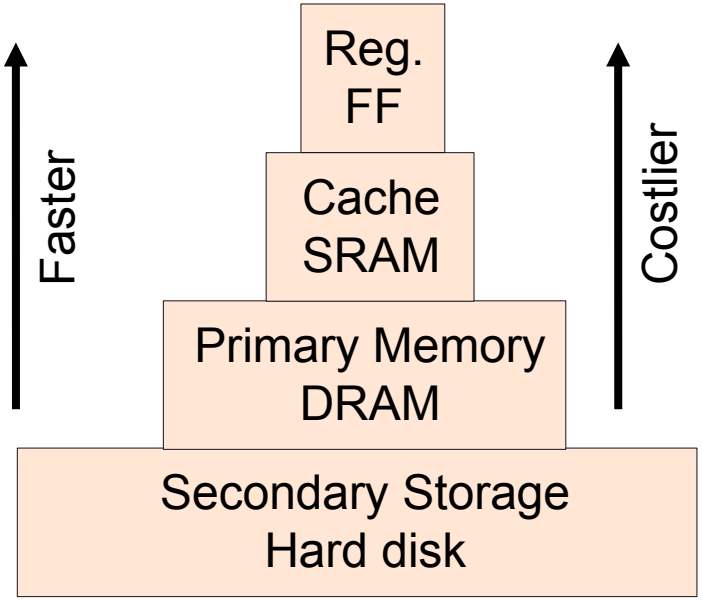










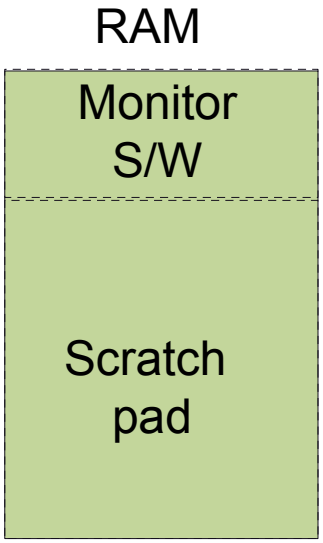


- Primary Memory Organization

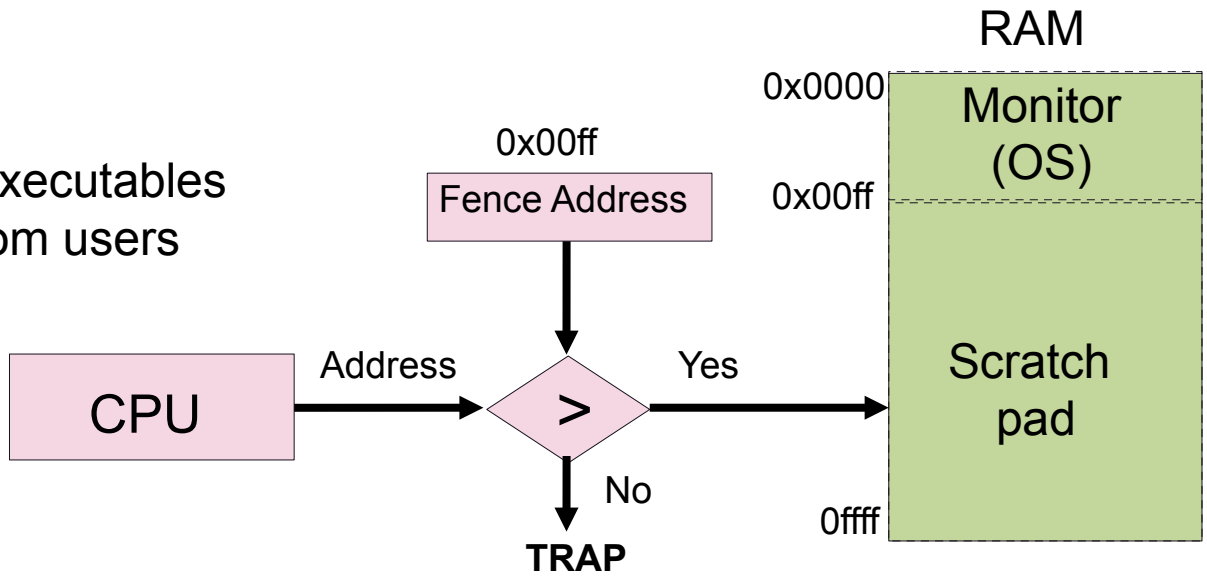
Primary Organization

Bare Machine
- Entire Mem. available to designer
- Flexible to designer
- Suitable for embedded systems
- Mem. organization efficient
- Not suitable for general purpose

RAM partitions:
- Monitor
  - System s/w
- Scratch pad
  - Temporary storage

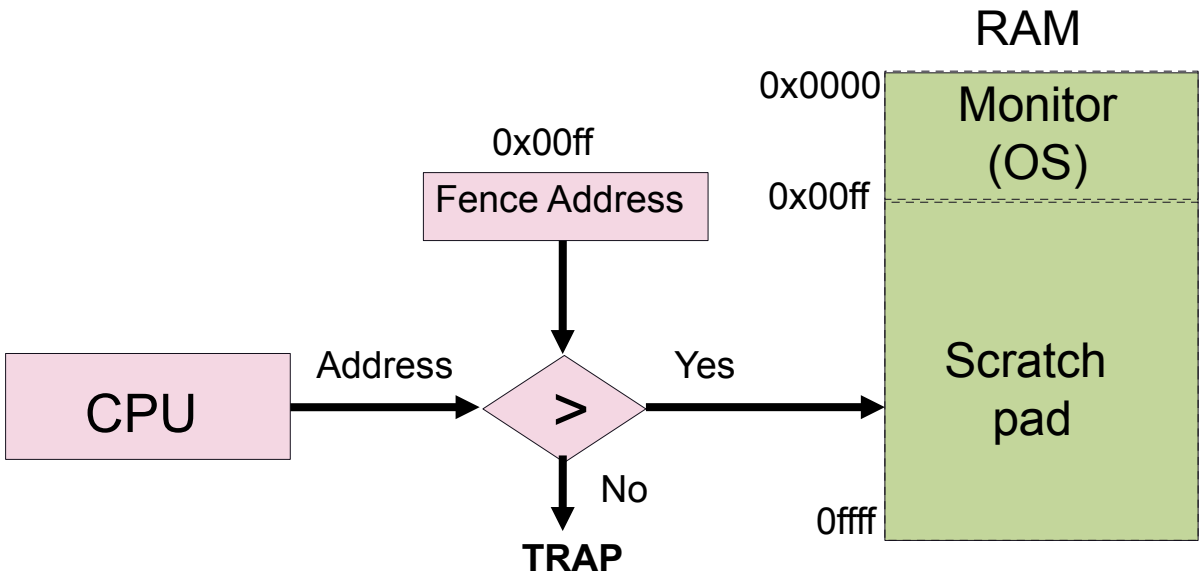


Resident monitor
- Designer doesn't know executables
- OS must be protected from users



Fence Implementation

Software solution (compiler)
- Check each address
- Pros
  - Flexible
- Cons (1 compr. for 1 addr)
  - More instr^{n}
  - Slow



Hardware solution (comparator)
- Check each address
- Pros
  - Fast
- Cons (fix fence addr)
  - OS can not be changed

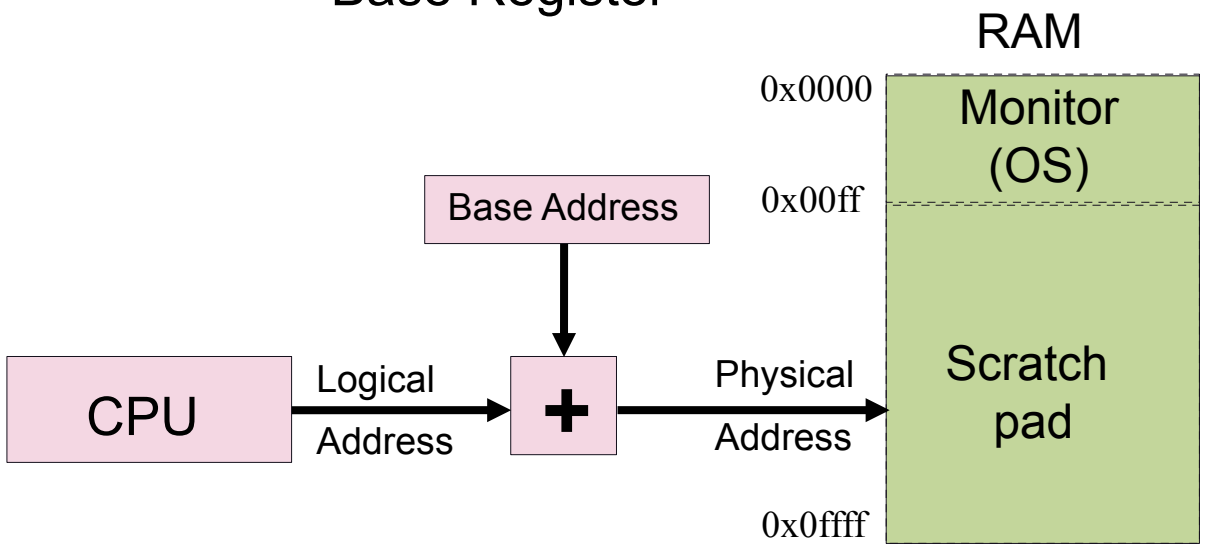
OS size 1MB Fence address 0x0fffff

OS size 2MB Fence address 0x1fffe

One option is to use a GPR for fence address

Fence Implementation

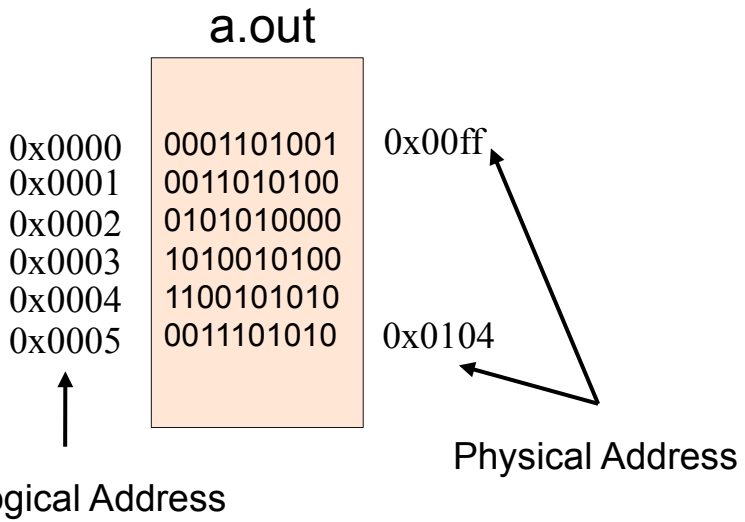
Base Register



PC start with 0x0000
Actual Mem. Addrs. PC + 0x00ff

How does it protect OS?





- Multiprogramming

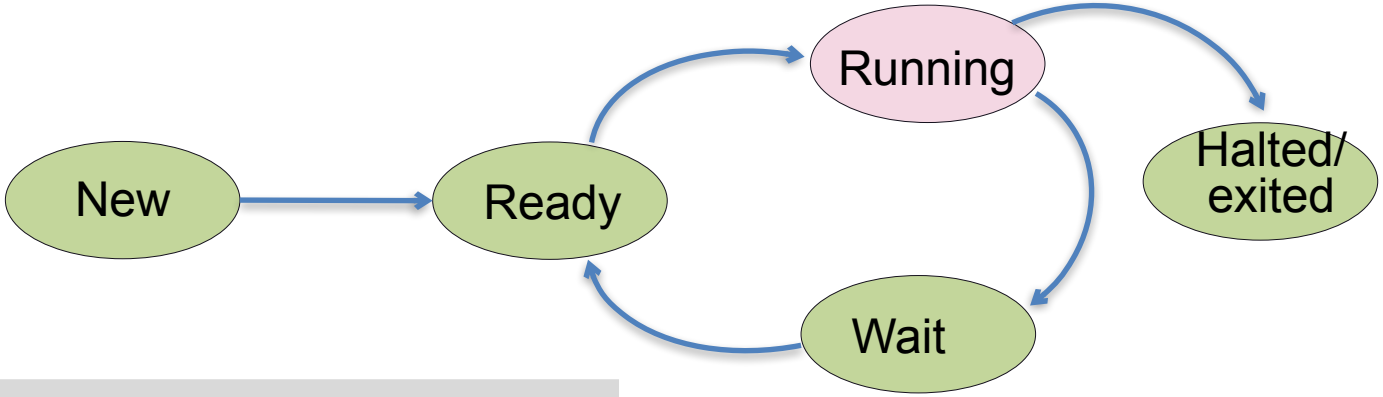
Multiprogramming Environment

Executes multiple programs together

hello

0001101001
0011010100
0101010000
1010010100
1100101010
0011101010

./hello



$gcc hello.c

Only one process executes in CPU

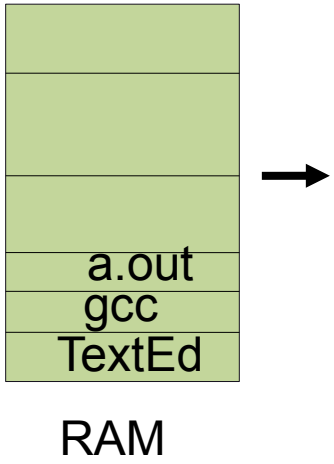
Process must be in RAM for execution

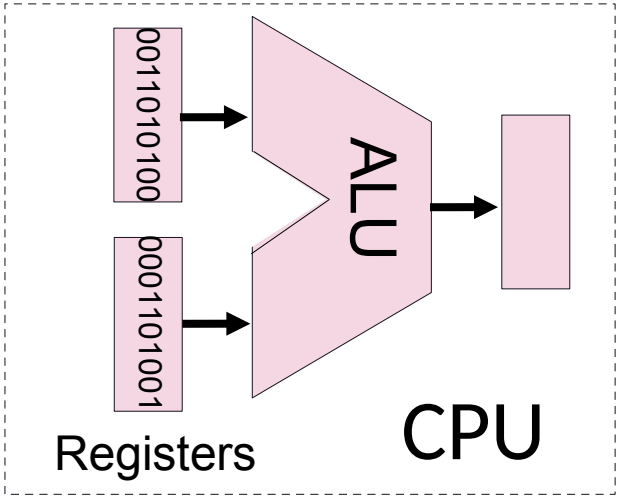
a.c

```
Void main()
{
int a ,b=9,c=8;
a=b+c
print(a)
}
```

Need:

- Ready queue
- Waiting queue

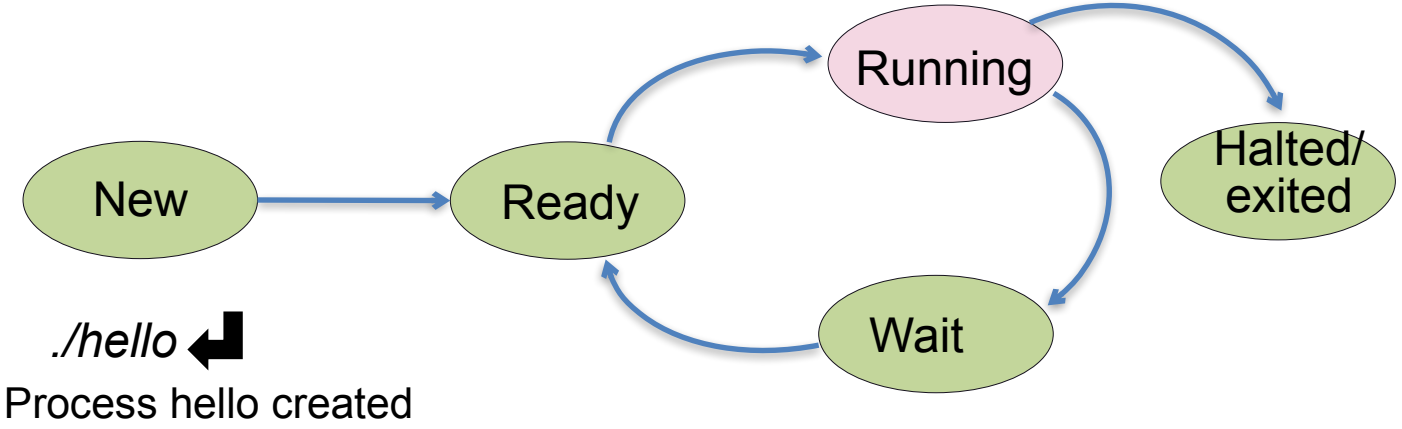


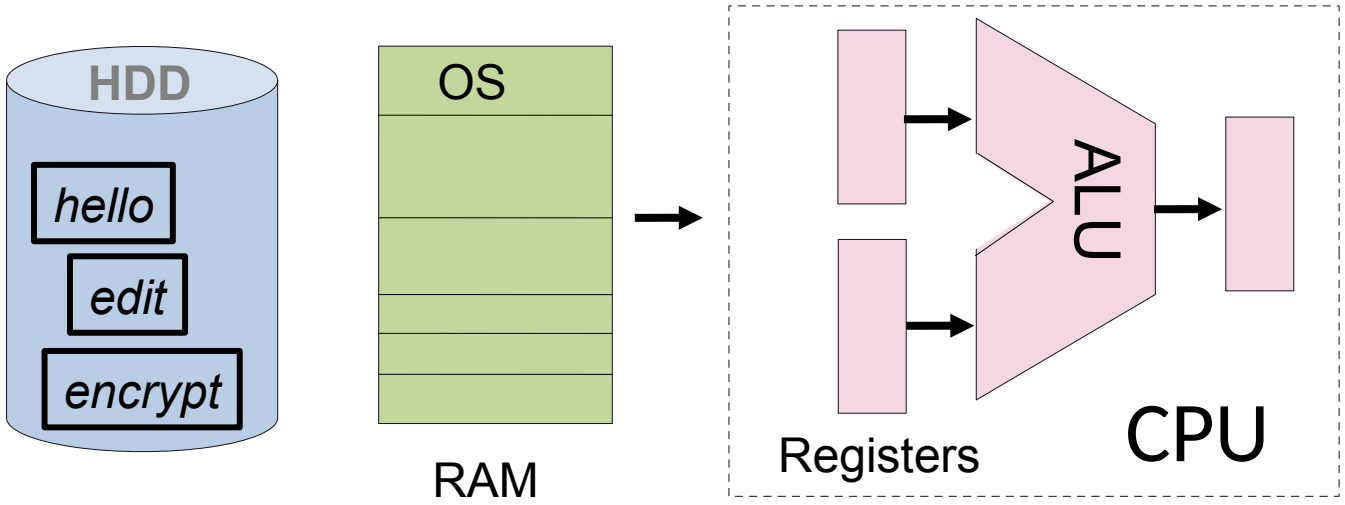


A running program is called a *process*

Multiprogramming Environment

Executes multiple programs together

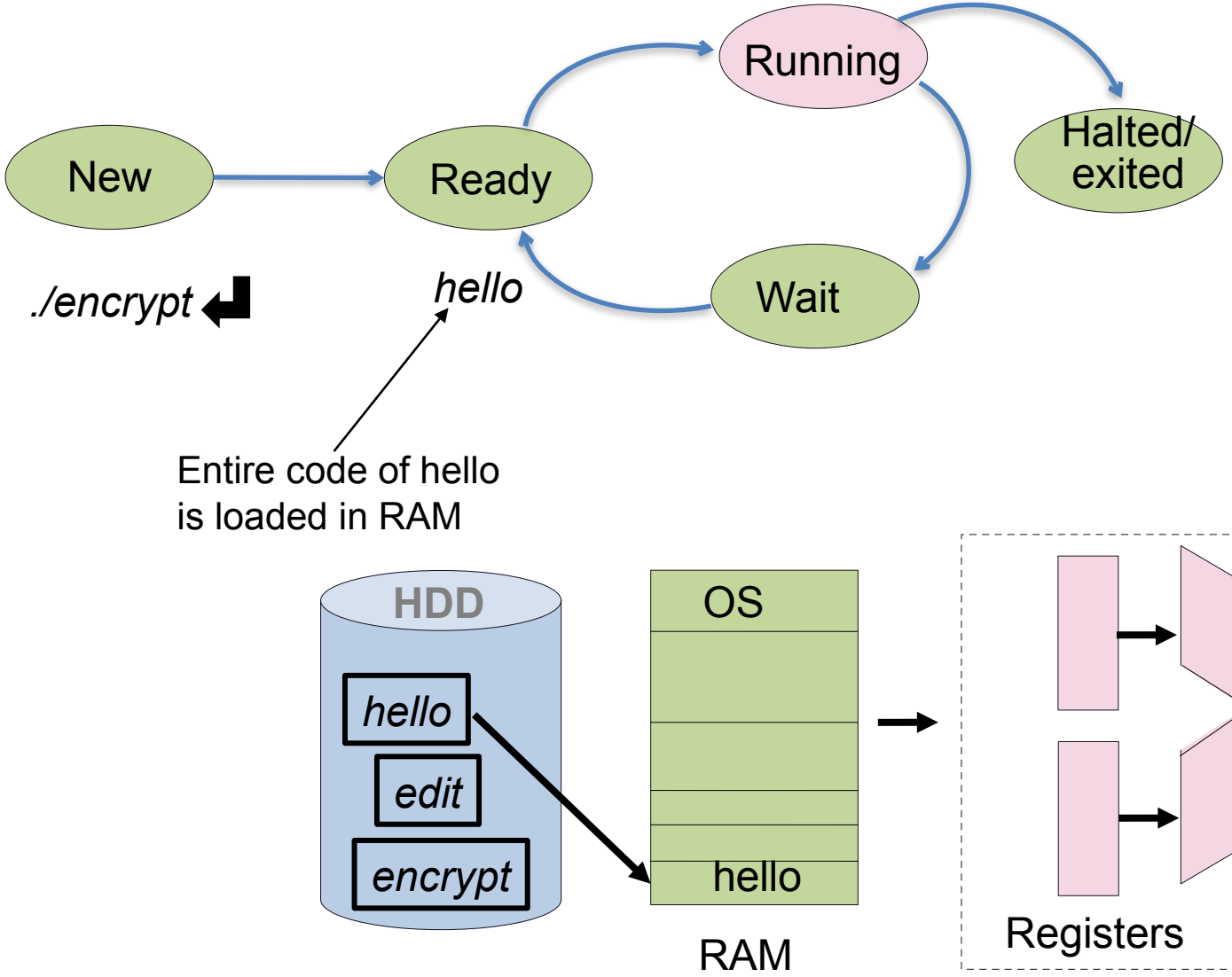




HDD : hard disc

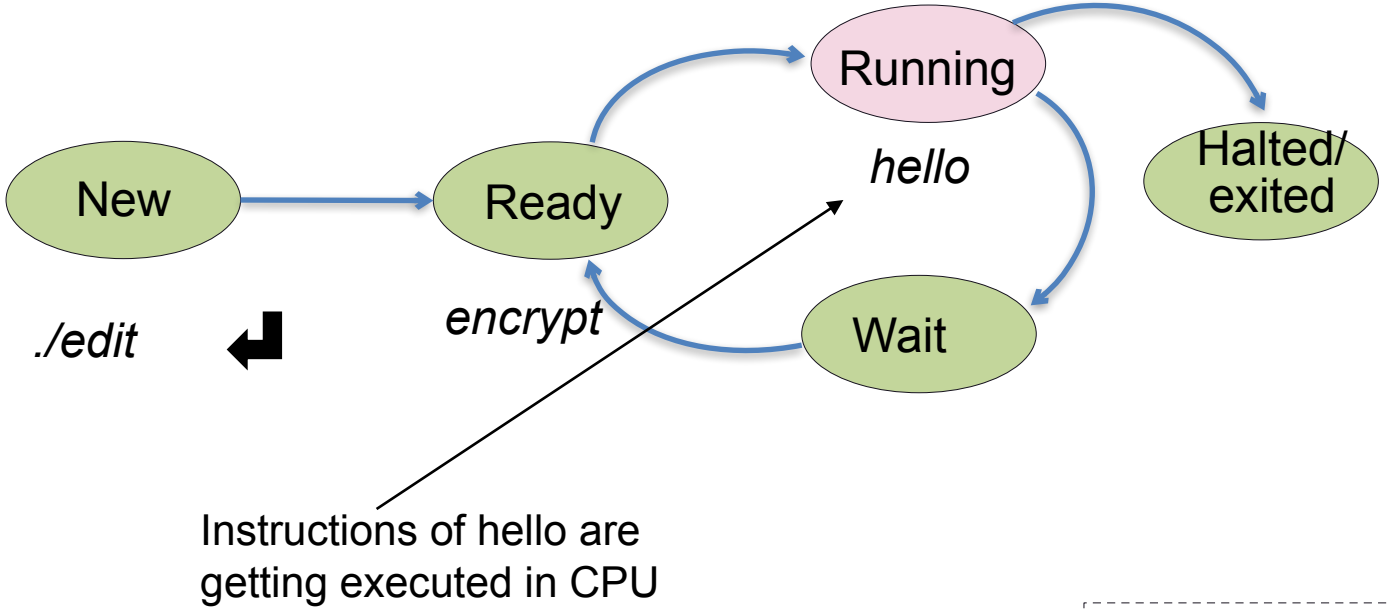
Multiprogramming Environment

Executes multiple programs together

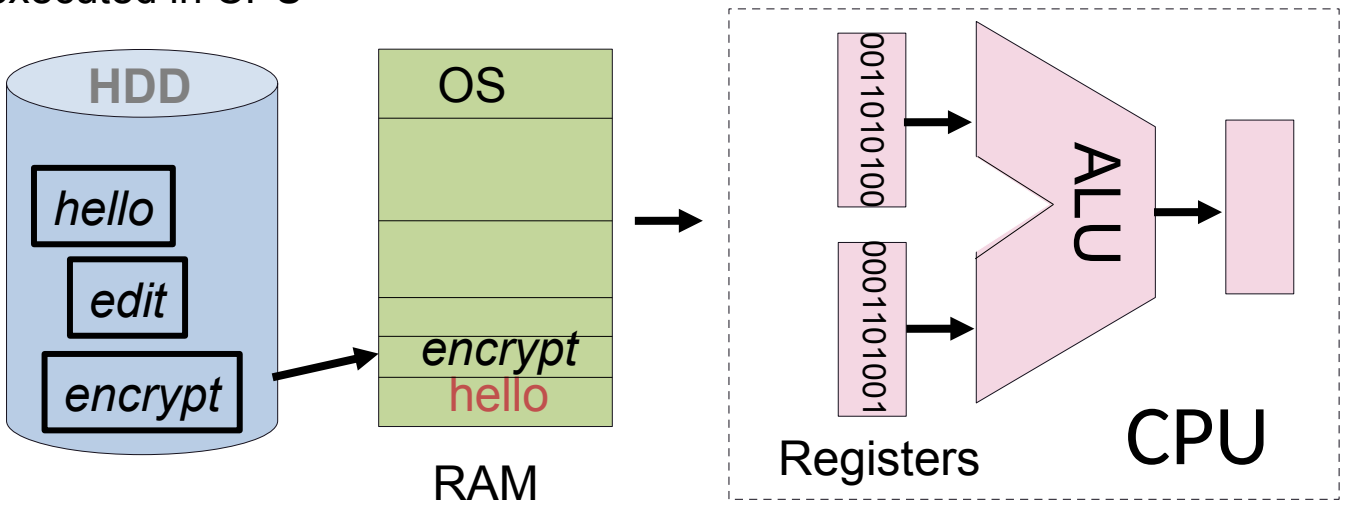


Multiprogramming Environment

Executes multiple programs together

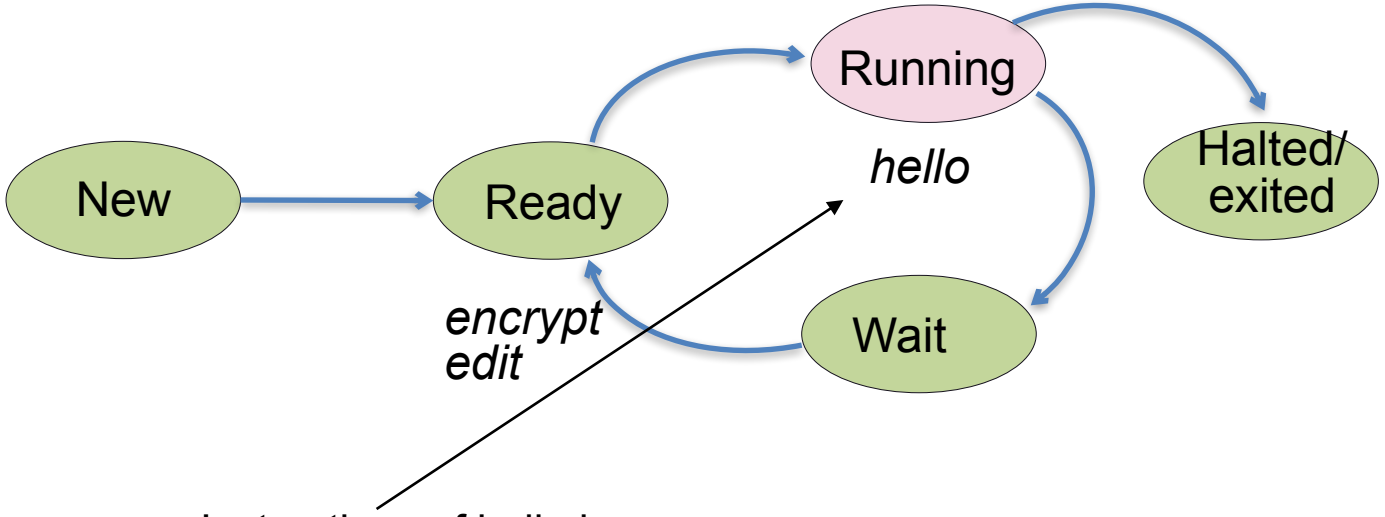


Instructions of hello are
getting executed in CPU



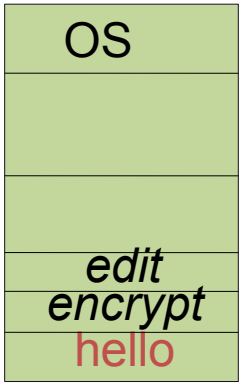
Multiprogramming Environment

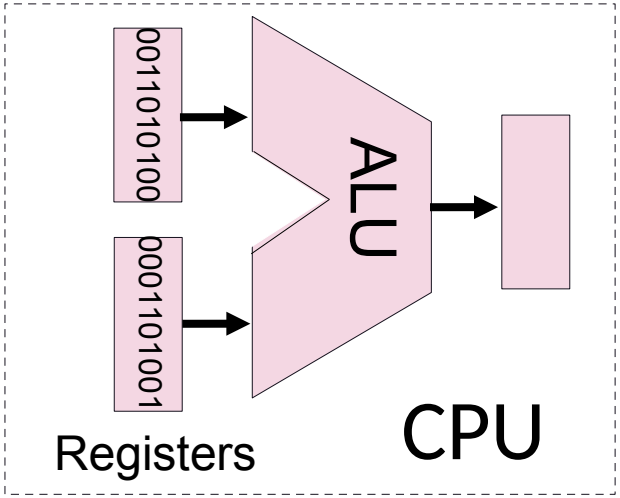
Executes multiple programs together




```
Void main()
{
int a= ,b=6,
    c=8;
scan(a)
c=a+b+c
print(a)
print(c)
}
```

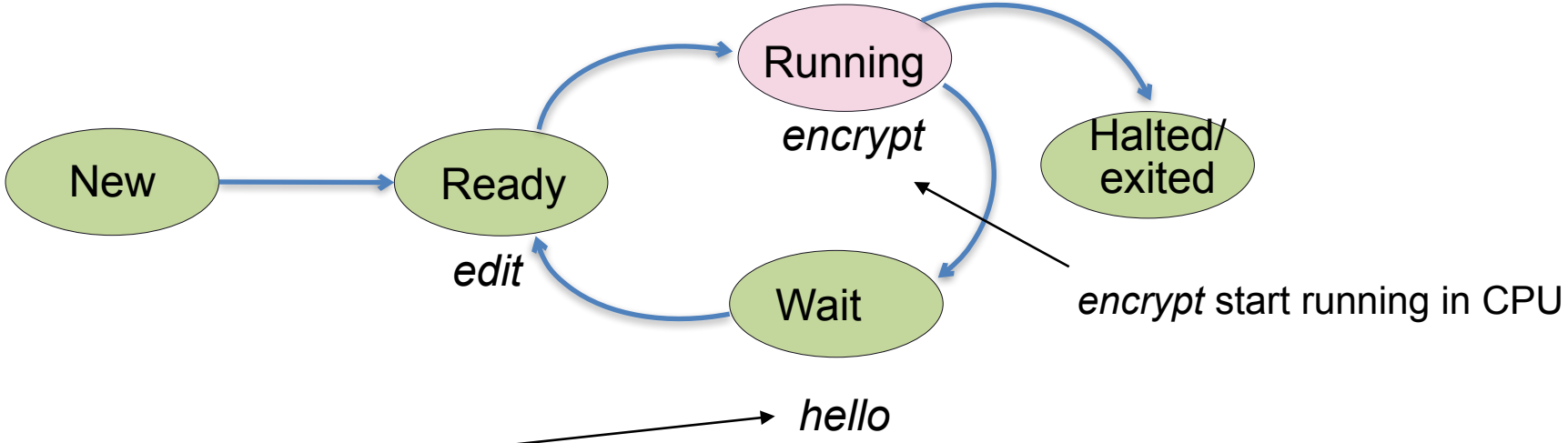
- I/O operations executes in I/O devices (slow)
- take more time





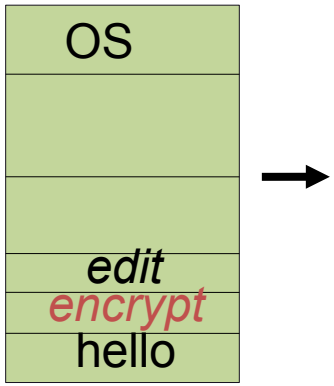
Multiprogramming Environment

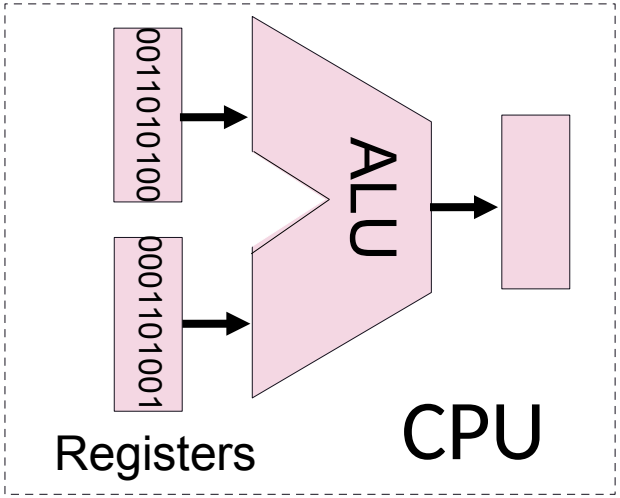
Executes multiple programs together




```
Void main()
{
int a= ,b=6,
    c=8;
scan(a)
c=a+b+c
print(a)
print(c)
}
```

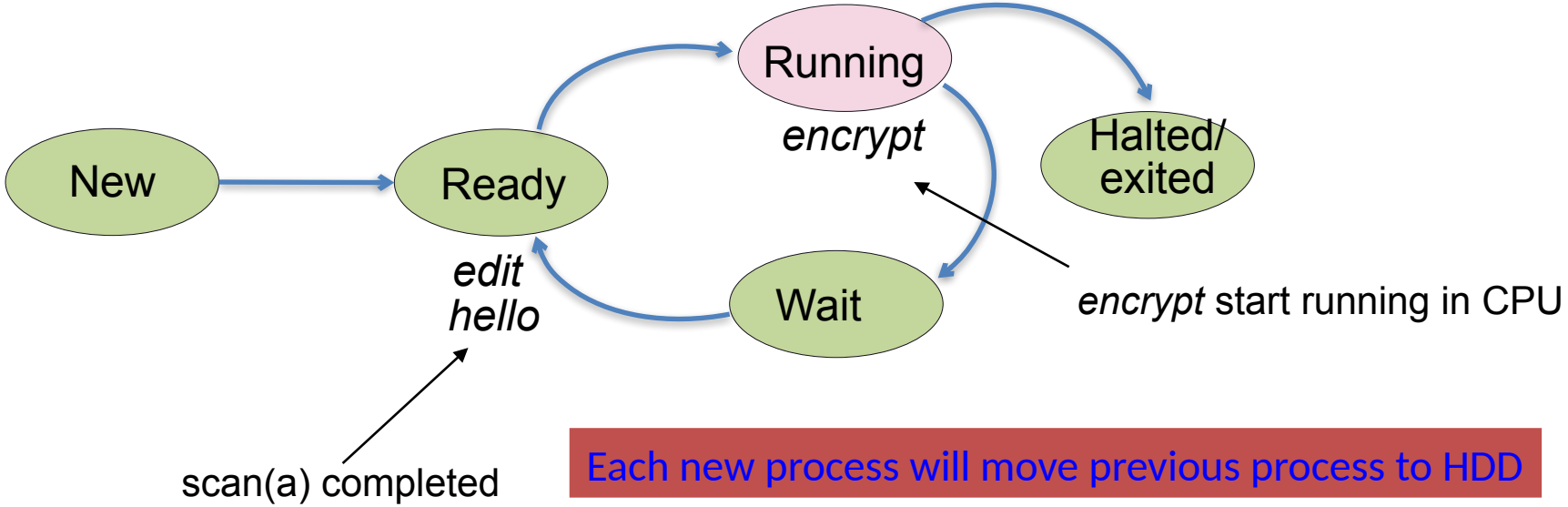
- I/O operations executes in I/O devices (slow)
- take more time





Multiprogramming Environment

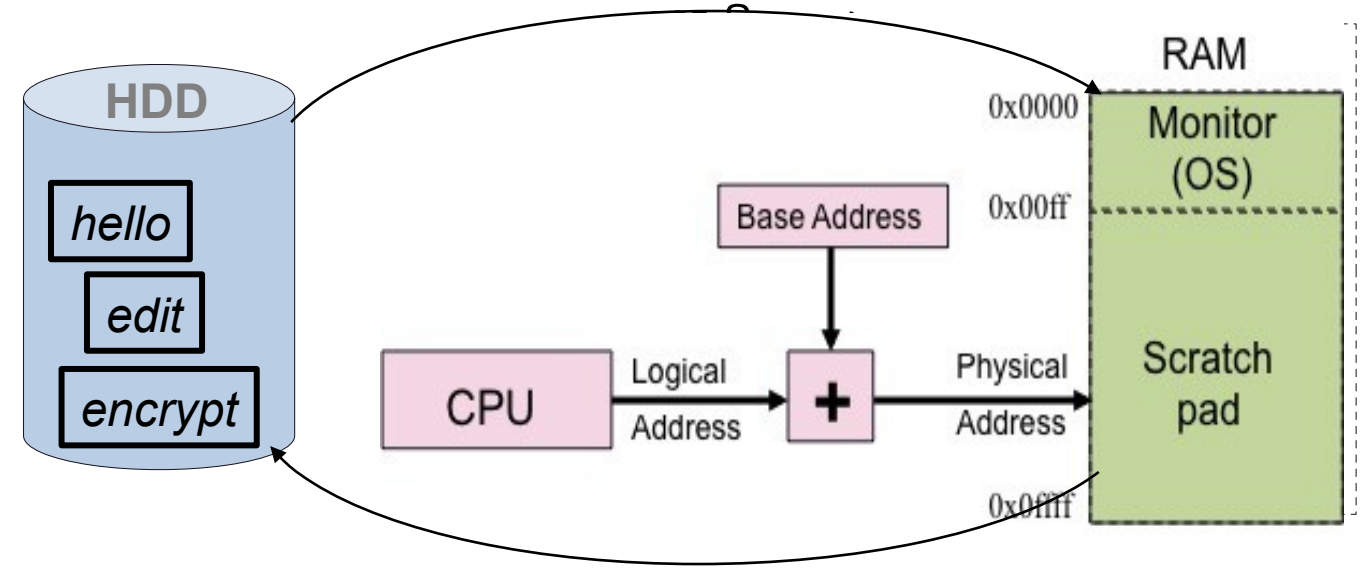
Executes multiple programs together



Each new process will move previous process to HDD

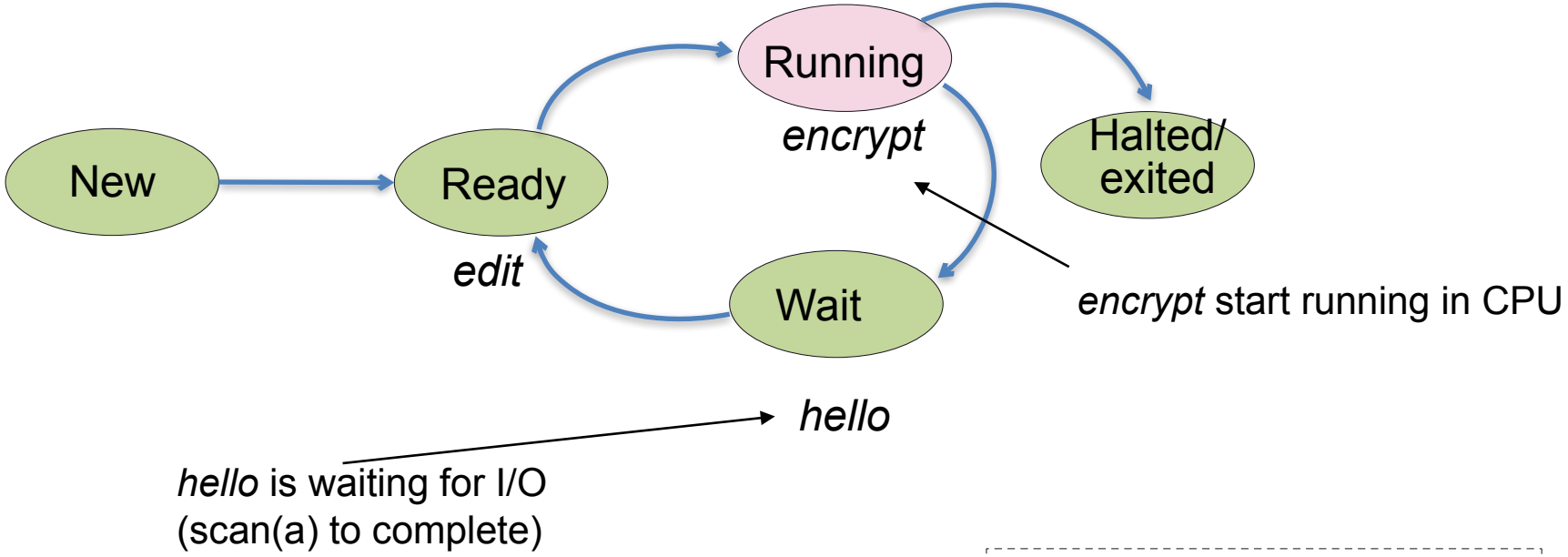
A lot of I/O

Slow processing

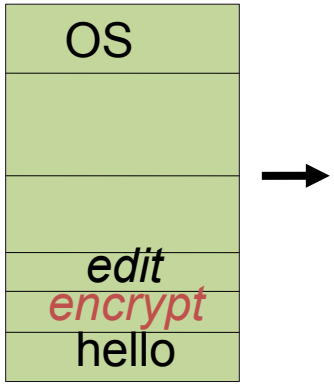


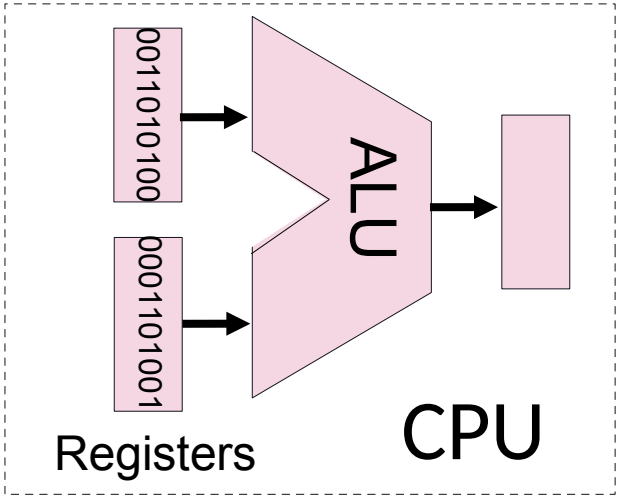
Multiprogramming Environment

Executes multiple programs together



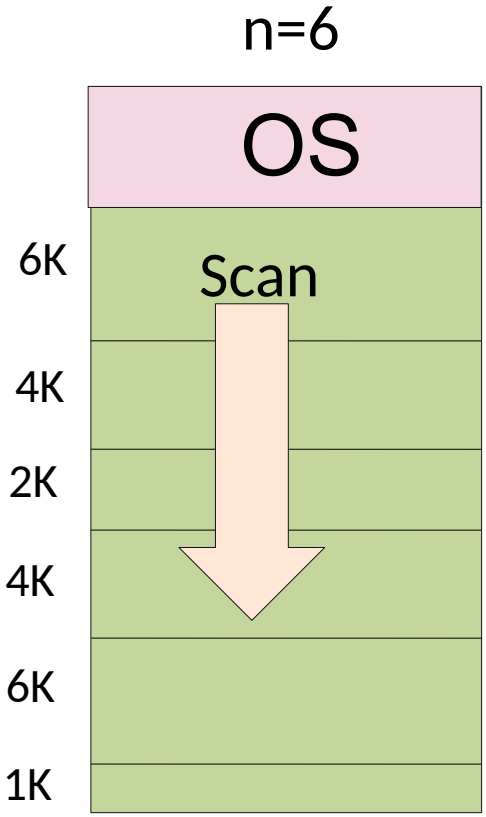
Memory should be partitioned for different processes





Fixed Memory Partitioning

Memory Partitioning



MFT: Multiprogramming with Fixed number of Tasks/Process

MFT can handle up to *n* number of processes

Loader: Load processes to different partitiones

There are multiple process allocation strategies

Strategy 1: First Fit

While loading a process in memory scan top to bottom:

- If a partition is available
- If the size of process is less than the partition
- Load the process

Fixed Memory Partitioning

Memory Partitioning



MFT: Multiprogramming with Fixed number of Tasks/Process

MFT can handle up to *n* number of processes

Loader: Load processes to different partitiones

There are multiple process allocation strategies

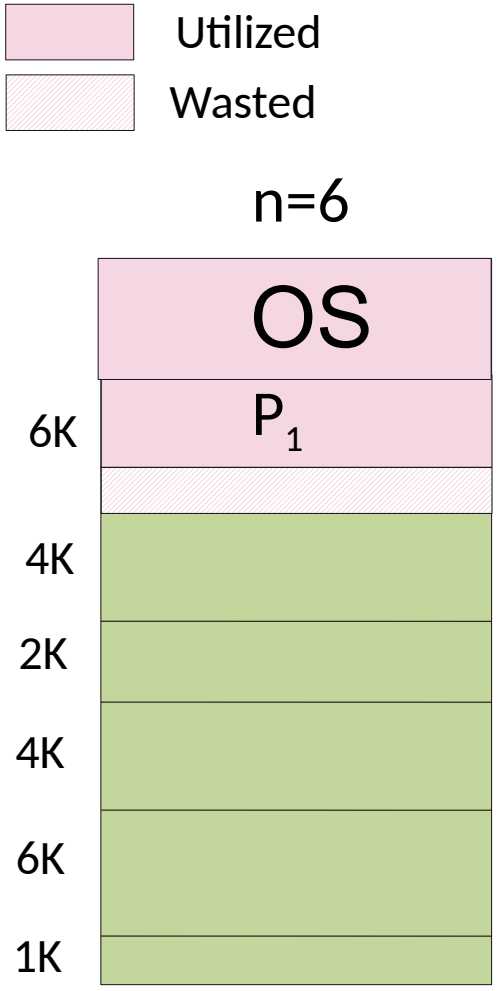
Strategy 1: First Fit

While loading a process in memory scan top to bottom:

- If a partition is available
- If the size of process is less than the partition
- Load the process

Process P_1 size is 4K

Fixed Memory Partitioning



Process P_1 size is 4K

Memory Partitioning

MFT: Multiprogramming with Fixed number of Tasks/Process

MFT can handle up to *n* number of processes

Loader: Load processes to different partitiones

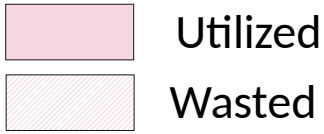
There are multiple process allocation strategies

Strategy 1: First Fit

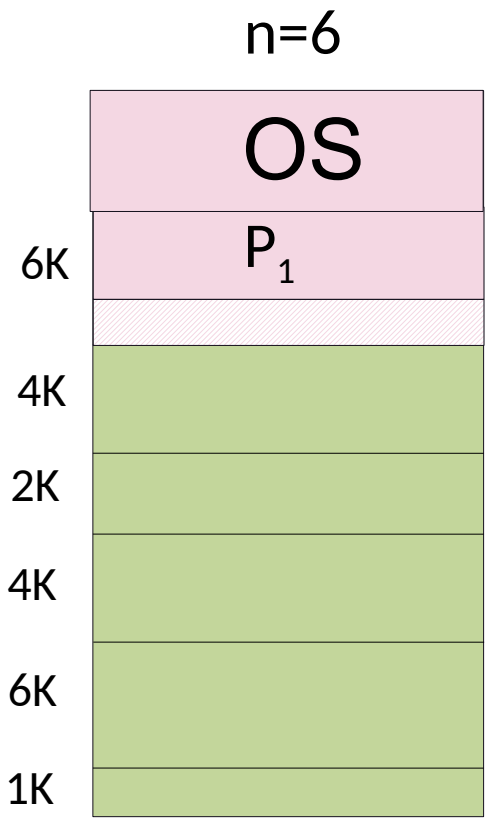
While loading a process in memory scan top to bottom:

- If a partition is available
- If the size of process is less than the partition
- Load the process

Fixed Memory Partitioning



Memory Partitioning



MFT: Multiprogramming with Fixed number of Tasks/Process

MFT can handle up to *n* number of processes

Loader: Load processes to different partitiones

There are multiple process allocation strategies

Strategy 1: First Fit

While loading a process in memory scan top to bottom:

- If a partition is available
- If the size of process is less than the partition
- Load the process

Process P_1 size is 4K

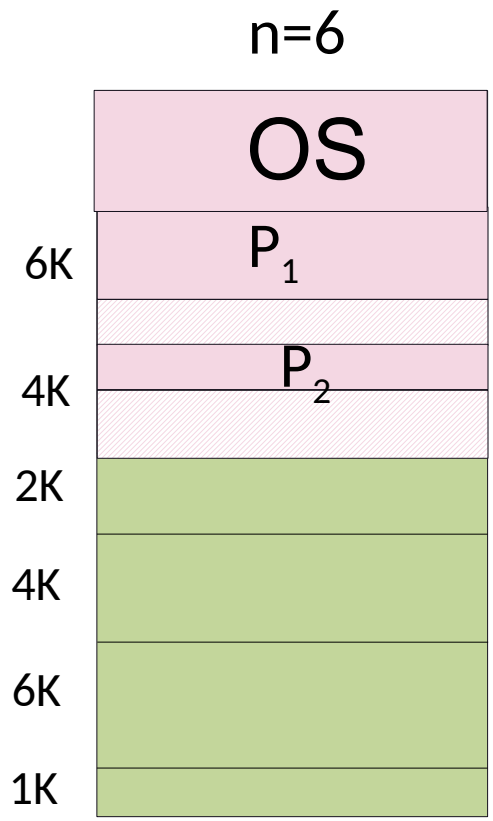
Process P_2 size is 1k

Fixed Memory Partitioning

Utilized

Wasted/fragmentation

Memory Partitioning



Process P_1 size is 4K

Process P_2 size is 1k

MFT: Multiprogramming with Fixed number of Tasks/Process

MFT can handle up to *n* number of processes

Loader: Load processes to different partitiones

There are multiple process allocation strategies

Strategy 1: First Fit

While loading a process in memory scan top to bottom:

- If a partition is available
- If the size of process is less than the partition
- Load the process

Pros

- Simple

Cons

- Wastage or fragmentation of memory

Internal fragmentation is high

Fixed Memory Partitioning

Memory Partitioning



MFT: Multiprogramming with Fixed number of Tasks/Process

MFT can handle up to *n* number of processes

Loader: Load processes to different partitiones

There are multiple process allocation strategies

Strategy 2: Best Fit

While loading a process in memory scan all:

- Choose the best partition
- Load the process

Process P_1 size is 4K

Fixed Memory Partitioning

Memory Partitioning



MFT: Multiprogramming with Fixed number of Tasks/Process

MFT can handle up to *n* number of processes

Loader: Load processes to different partitiones

There are multiple process allocation strategies

Strategy 2: Best Fit

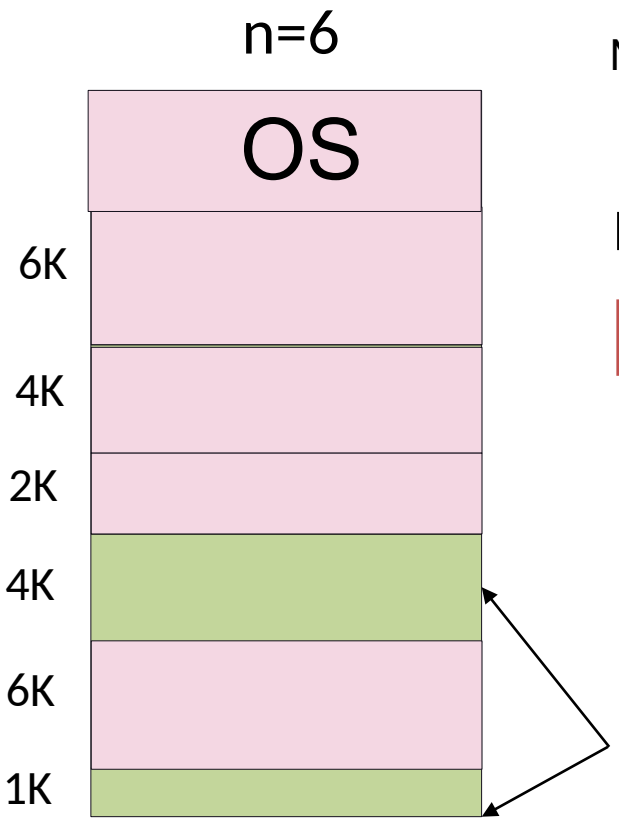
While loading a process in memory scan all:

- Choose the best partition
- Load the process

Process P_5 size is 5K

Fixed Memory Partitioning

Memory Partitioning



MFT: Multiprogramming with Fixed number of Tasks/Process

MFT can handle up to *n* number of processes

Loader: Load processes to different partitiones

There are multiple process allocation strategies

Strategy 2: Best Fit

While loading a process in memory scan all:

- Choose the best partition
- Load the process

External fragmentation

Process P_5 size is 5K

Fixed Memory Partitioning

Memory Partitioning



Process P_5 size is 5K

MFT: Multiprogramming with Fixed number of Tasks/Process

MFT can handle up to *n* number of processes

Loader: Load processes to different partitiones

There are multiple process allocation strategies

Strategy 2: Best Fit

While loading a process in memory scan all:

- Choose the best partition
- Load the process

External fragmentation

Pros

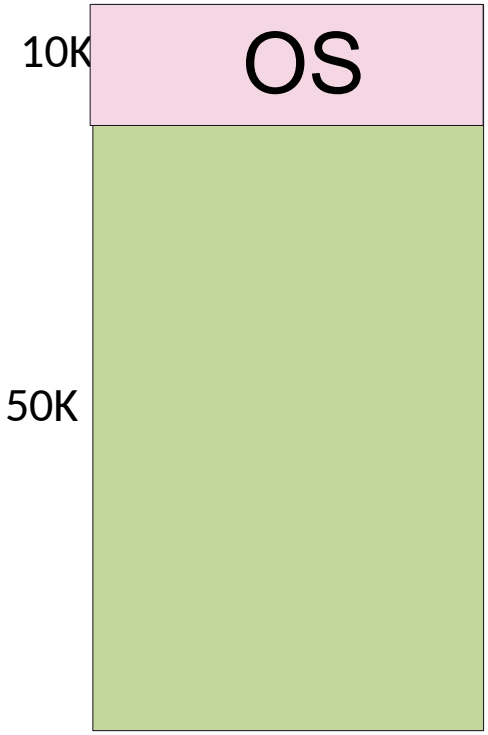
- Less internal fragmentation

Cons

- Complex
- External fragmentation

Variable Memory Partitioning

Memory Partitioning



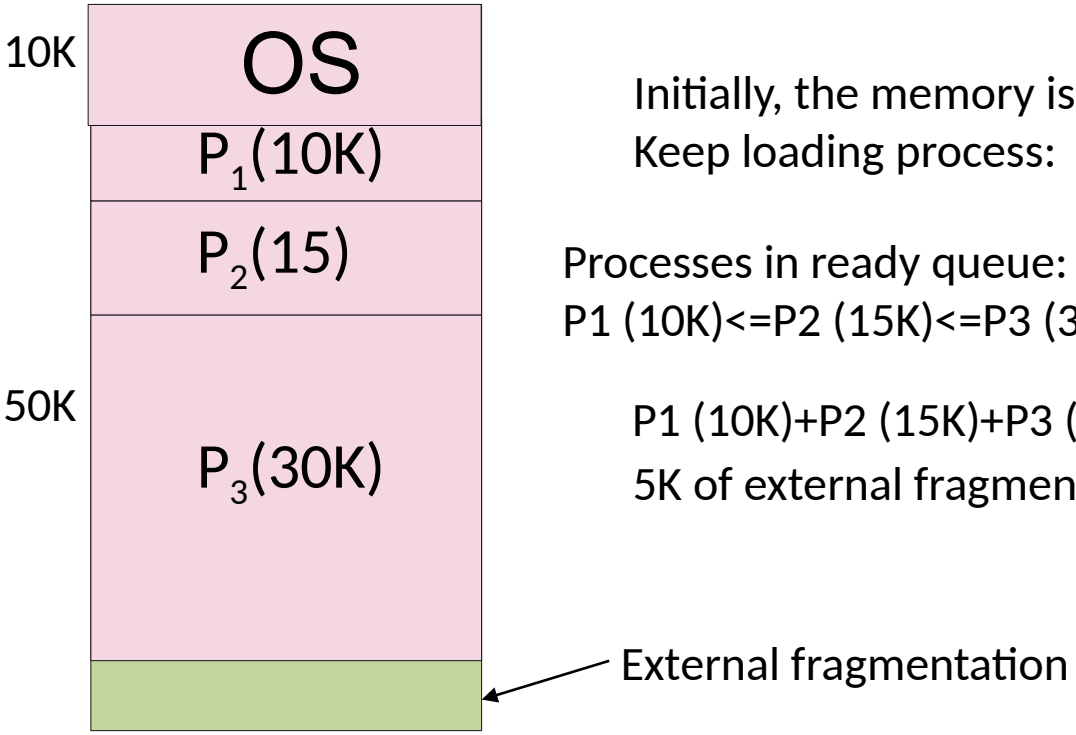
MVT: Multiprogramming with variable number of Tasks/Process

Initially, the memory is one segment
Keep loading process:

Processes in ready queue:
P1 (10K)<=P2 (15K)<=P3 (30K)<=P4 (20K)

Fixed Memory Partitioning

Memory Partitioning



MVT: Multiprogramming with variable number of Tasks/Process

Initially, the memory is one segment
Keep loading process:

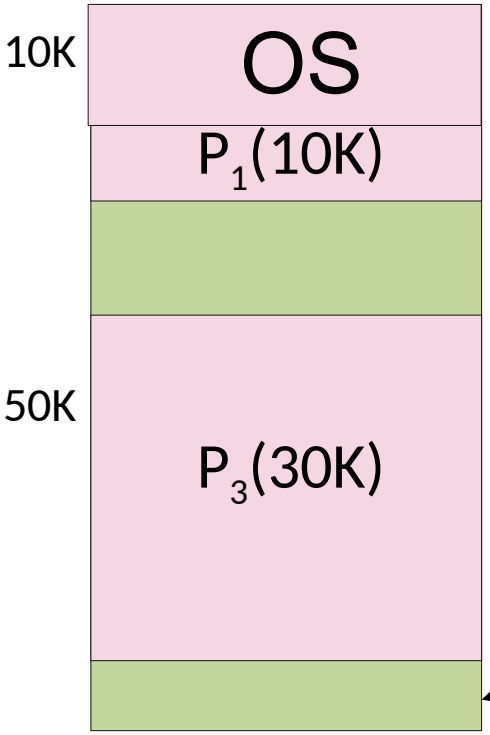
Processes in ready queue:
P1 (10K)<=P2 (15K)<=P3 (30K)<=P4 (20K)

P1 (10K)+P2 (15K)+P3 (30K)=45K

5K of external fragmentation

Fixed Memory Partitioning

Memory Partitioning



MVT: Multiprogramming with variable number of Tasks/Process

Initially, the memory is one segment
Keep loading process:

Processes in ready queue:
P1 (10K)<=P2 (15K)<=P3 (30K)<=P4 (20K)

P1 (10K)+P2 (15K)+P3 (30K)=45K
5K of external fragmentation

P2 completes

20K free space still P4 can not run

External fragmentation

Pros

- No internal fragmentation

Cons

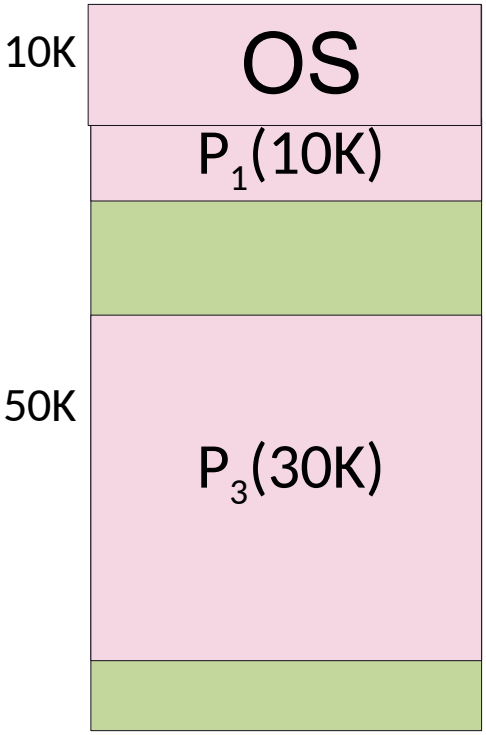
- Complex implementation
- External fragmentation

Fixed Memory Partitioning

Memory Partitioning

Compaction

Shift existing processes byte-by-byte up or down to make a big partition

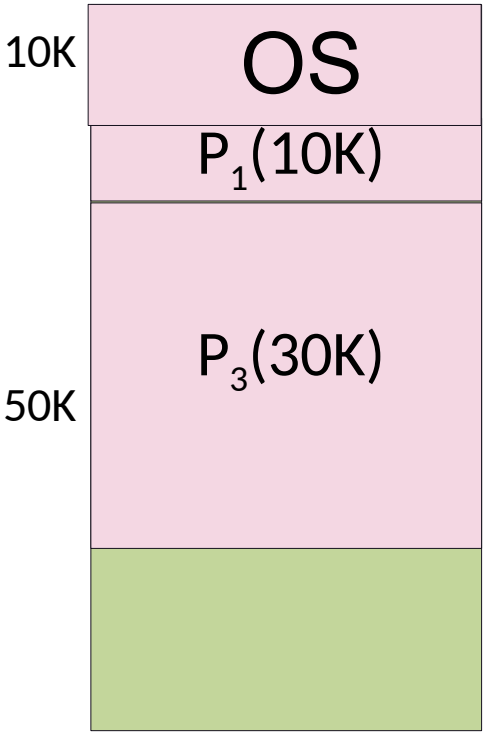


Fixed Memory Partitioning

Memory Partitioning

Compaction

Shift existing processes byte-by-byte up or down to make a big partition



Compaction can overcome external fragmentation up to certain extent

How to keep track of partitions

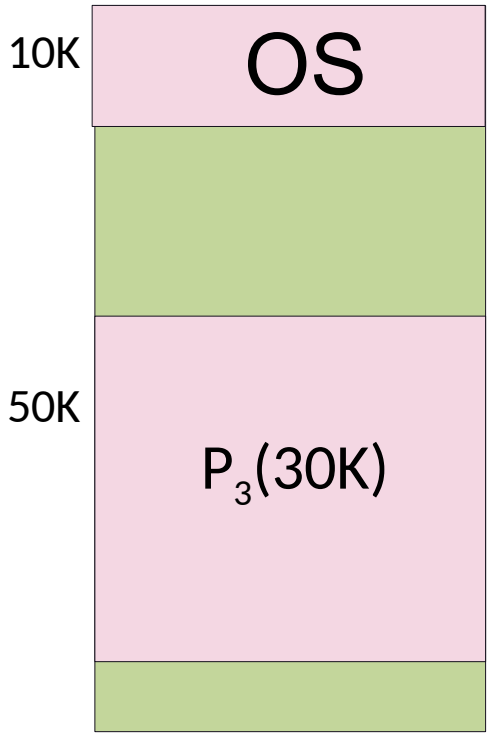
Maintaining a table will waste memory

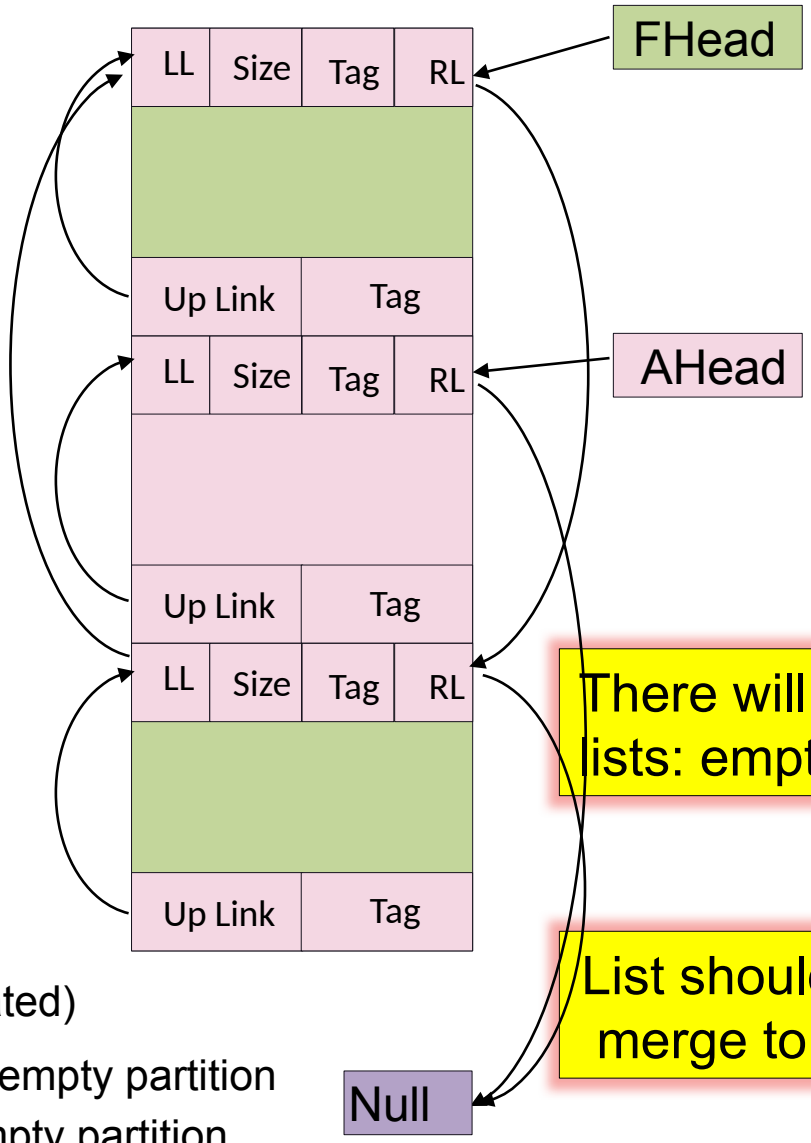
Use link list

Fixed Memory Partitioning

Memory Partitioning

Head: Points the first partition





There will be two link lists: empty and allocated

List should have option to merge to empty partition

Size: Size of the partition

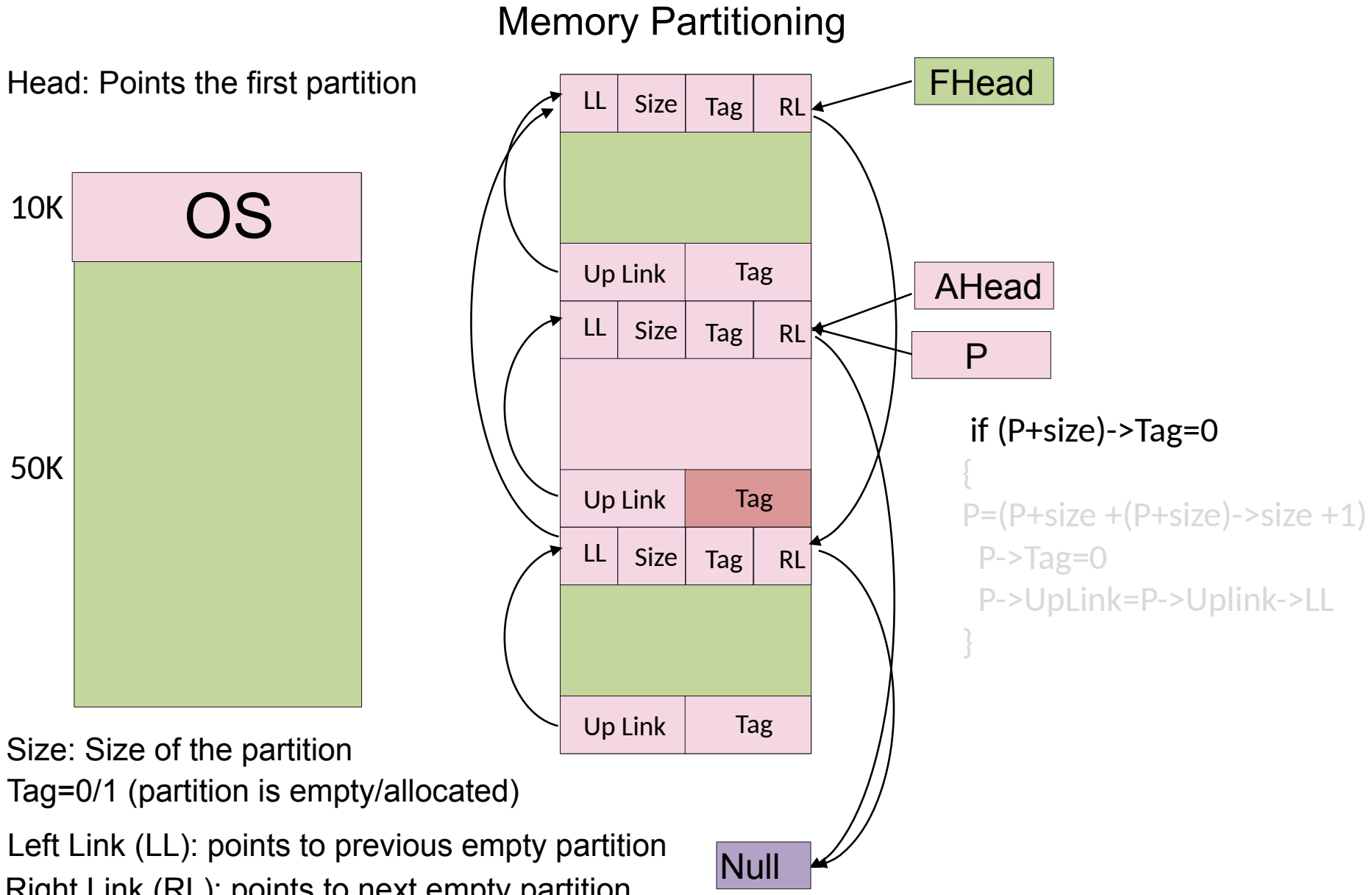
Tag=0/1 (partition is empty/allocated)

Left Link (LL): points to previous empty partition

Right Link (RL): points to next empty partition

Up Link : points to its own LL

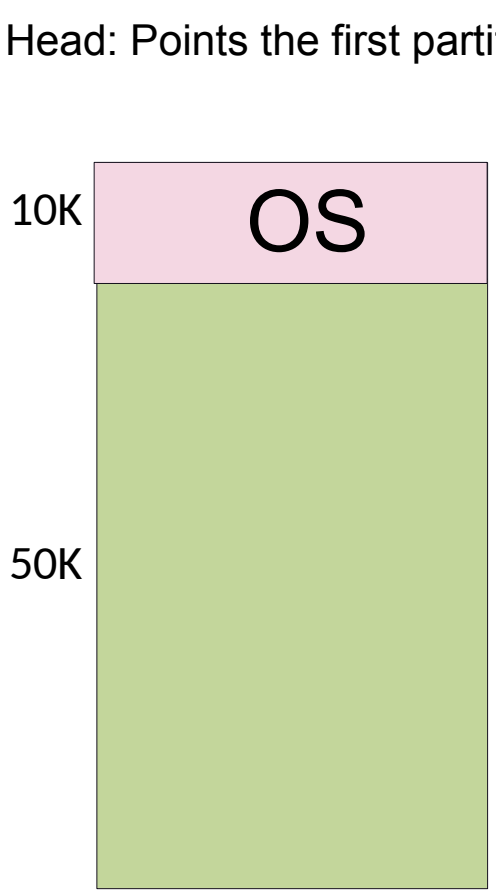
Fixed Memory Partitioning

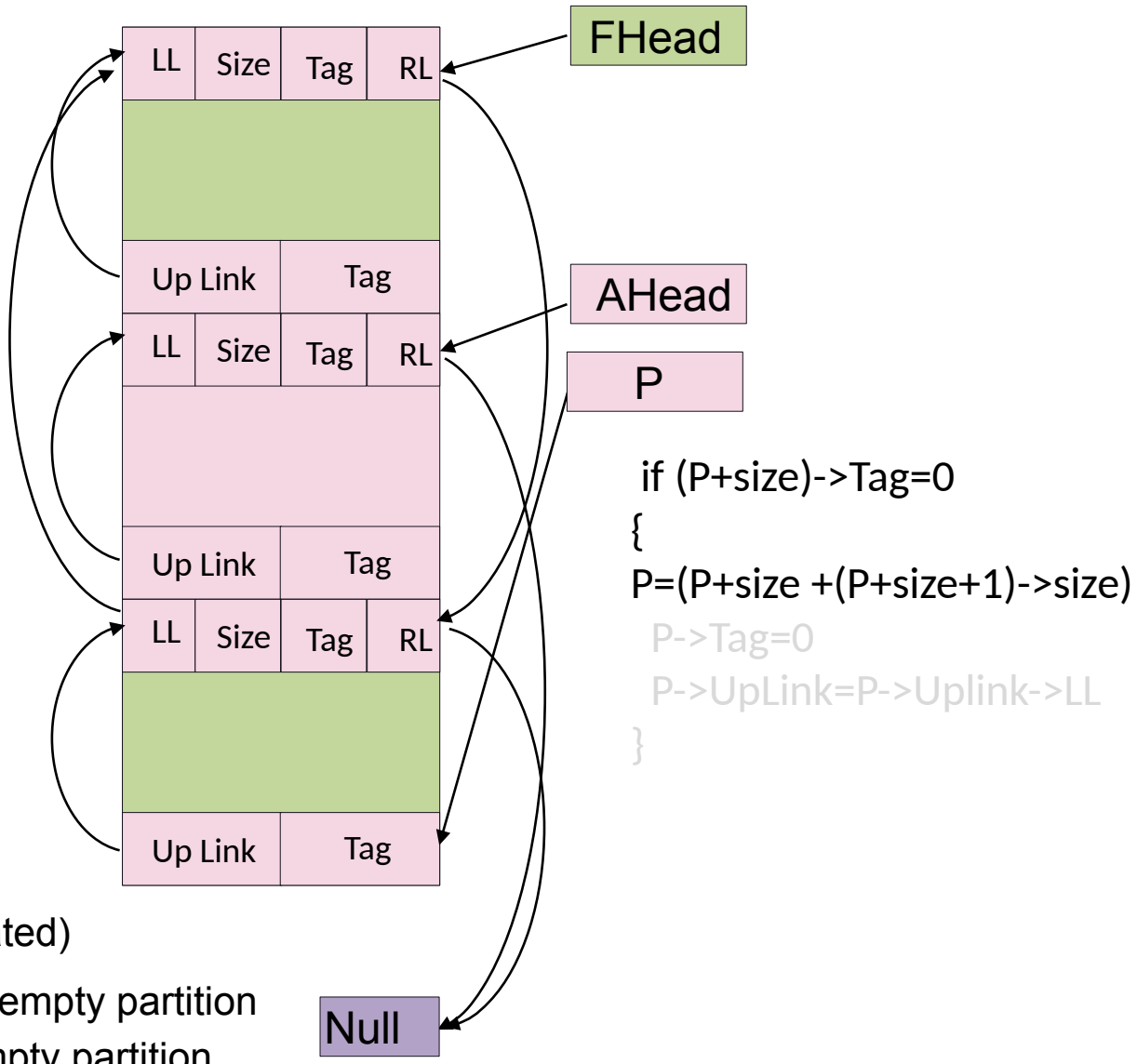


Fixed Memory Partitioning

Memory Partitioning

Head: Points the first partition






```
if (P+size)->Tag=0
{
P=(P+size +(P+size+1)->size)
  P->Tag=0
  P->UpLink=P->Uplink->LL
}
```

Size: Size of the partition

Tag=0/1 (partition is empty/allocated)

Left Link (LL): points to previous empty partition

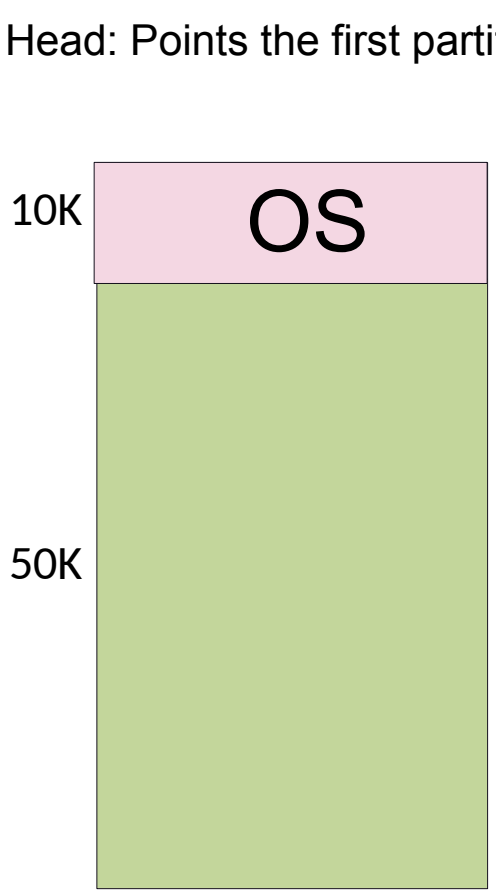
Right Link (RL): points to next empty partition

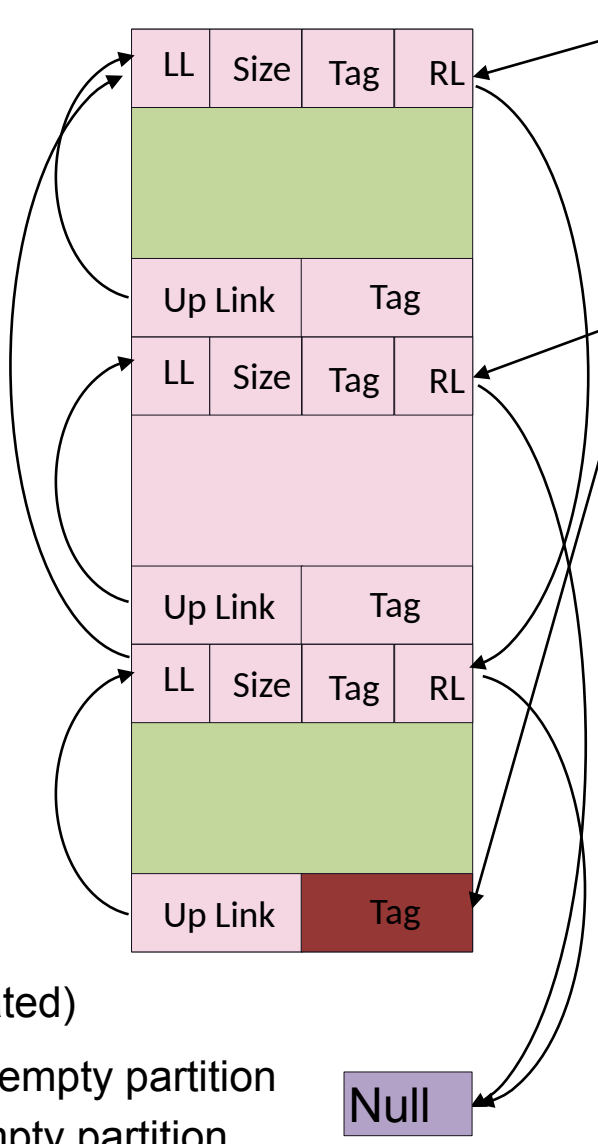
Up Link : points to its own LL

Fixed Memory Partitioning

Memory Partitioning

Head: Points the first partition






```
if (P+size)->Tag=0
{
P=(P+size +(P+size+1)->size)
 P->Tag=0
 P->UpLink=P->Uplink->LL
}
```

Size: Size of the partition

Tag=0/1 (partition is empty/allocated)

Left Link (LL): points to previous empty partition

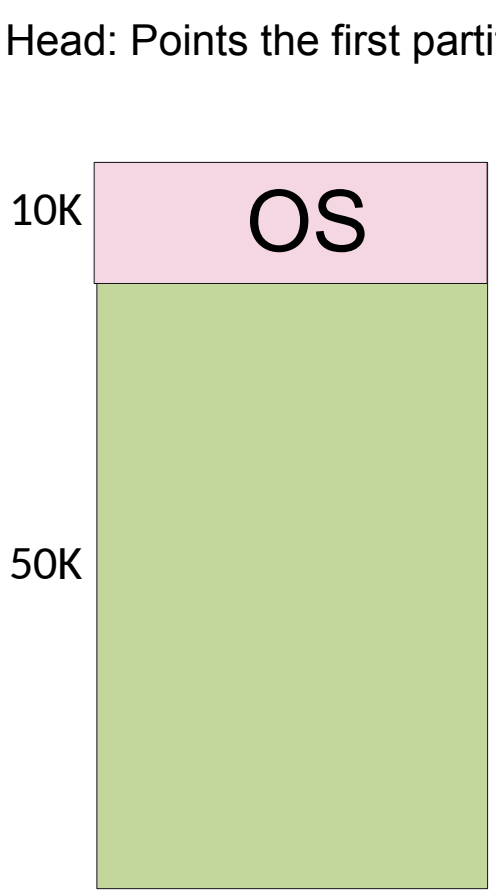
Right Link (RL): points to next empty partition

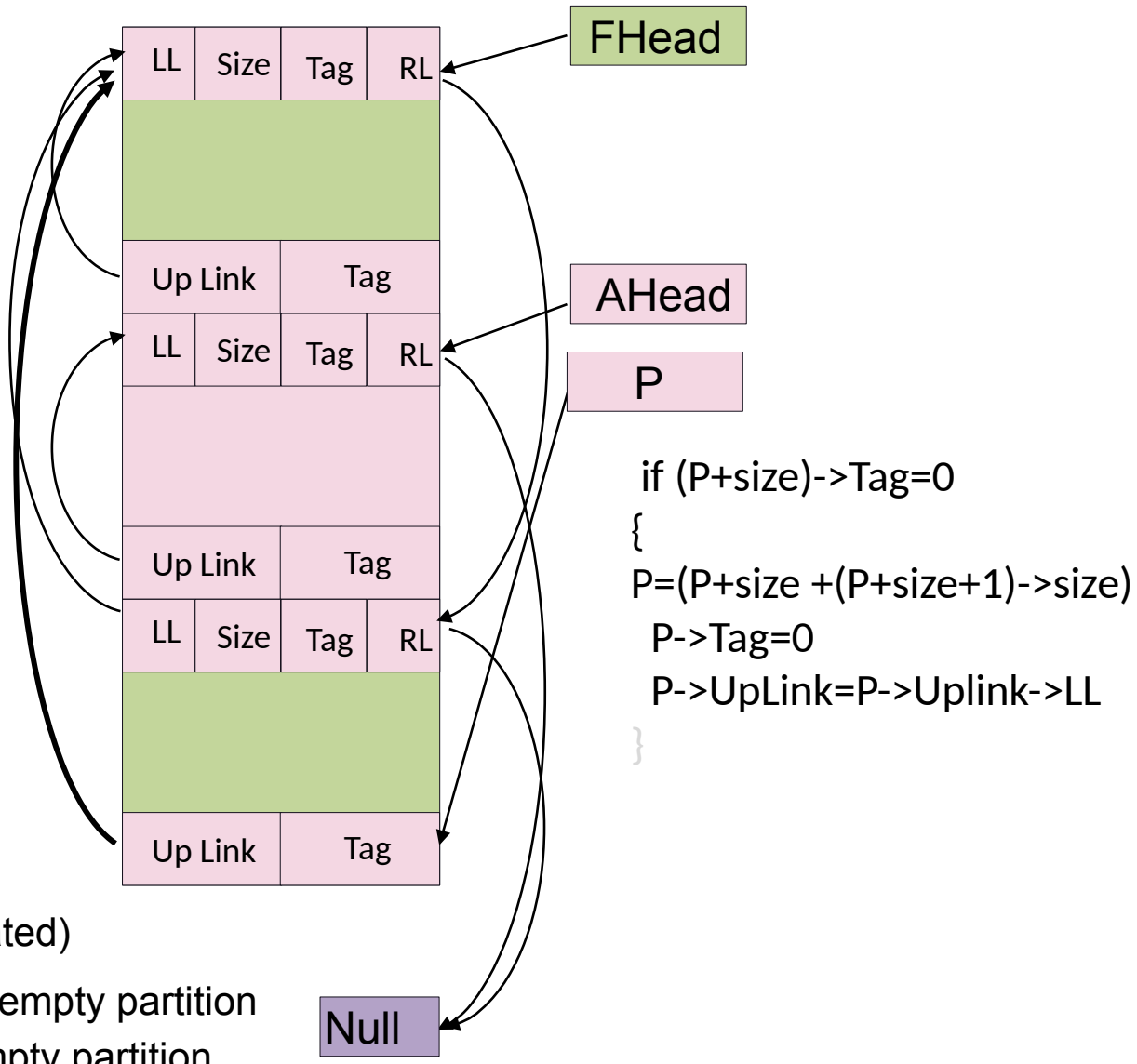
Up Link : points to its own LL

Fixed Memory Partitioning

Memory Partitioning

Head: Points the first partition






```
if (P+size)->Tag=0
{
P=(P+size +(P+size+1)->size)
  P->Tag=0
  P->UpLink=P->Uplink->LL
}
```

Size: Size of the partition

Tag=0/1 (partition is empty/allocated)

Left Link (LL): points to previous empty partition

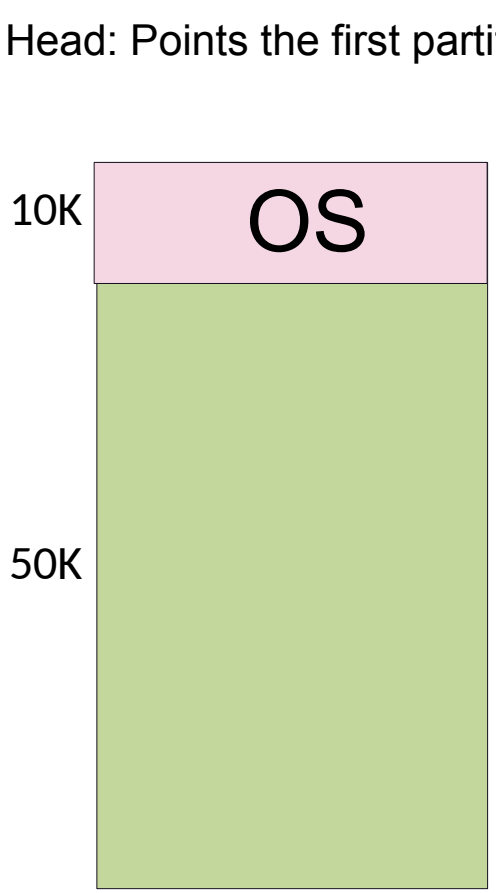
Right Link (RL): points to next empty partition

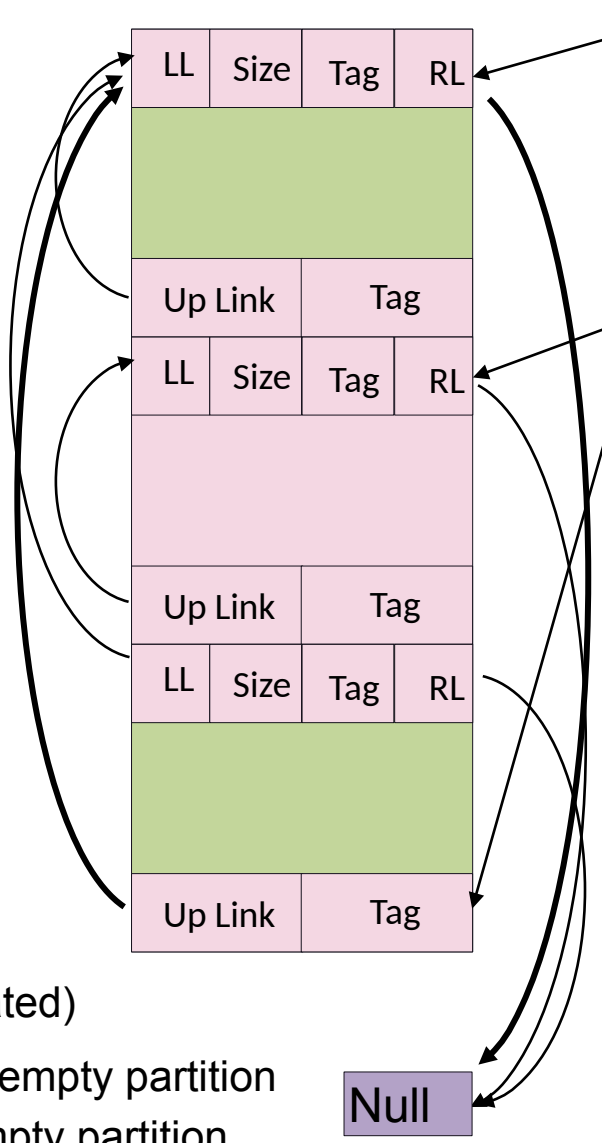
Up Link : points to its own LL

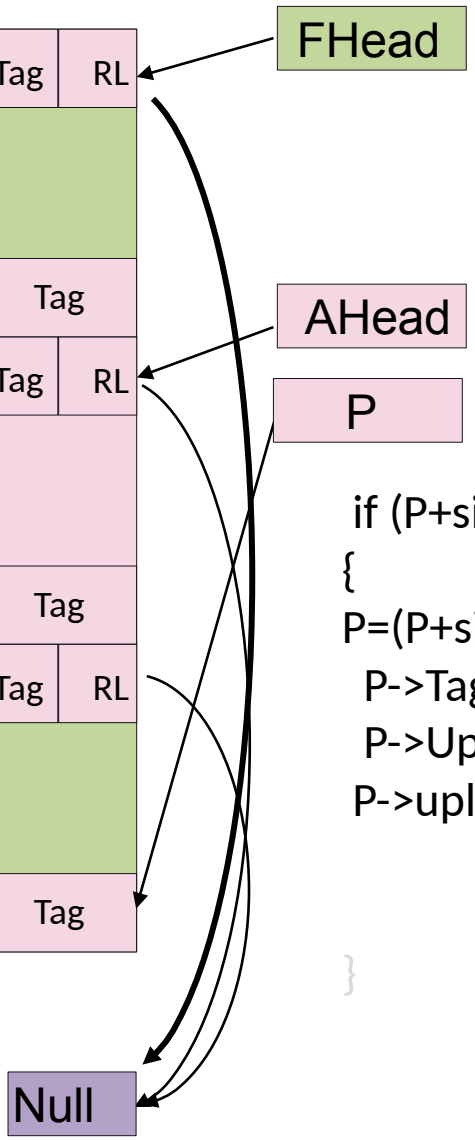
Fixed Memory Partitioning

Memory Partitioning

Head: Points the first partition








```
if (P+size)->Tag=0
{
P=(P+size +(P+size+1)->size)
  P->Tag=0
  P->UpLink=P->Uplink->LL
 P->uplink->RL=
                P->uplink->RL->RL

}
```

Size: Size of the partition

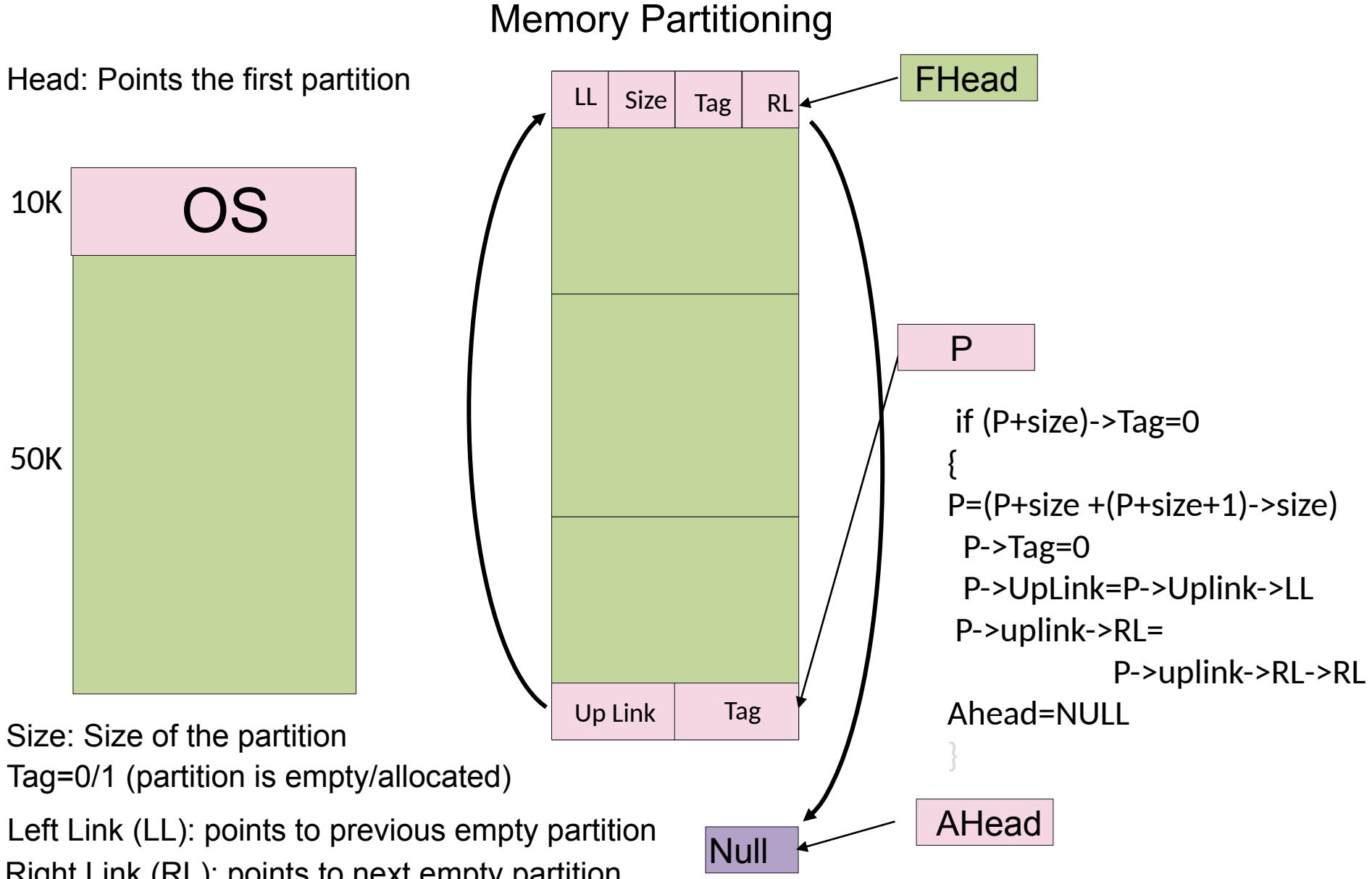
Tag=0/1 (partition is empty/allocated)

Left Link (LL): points to previous empty partition

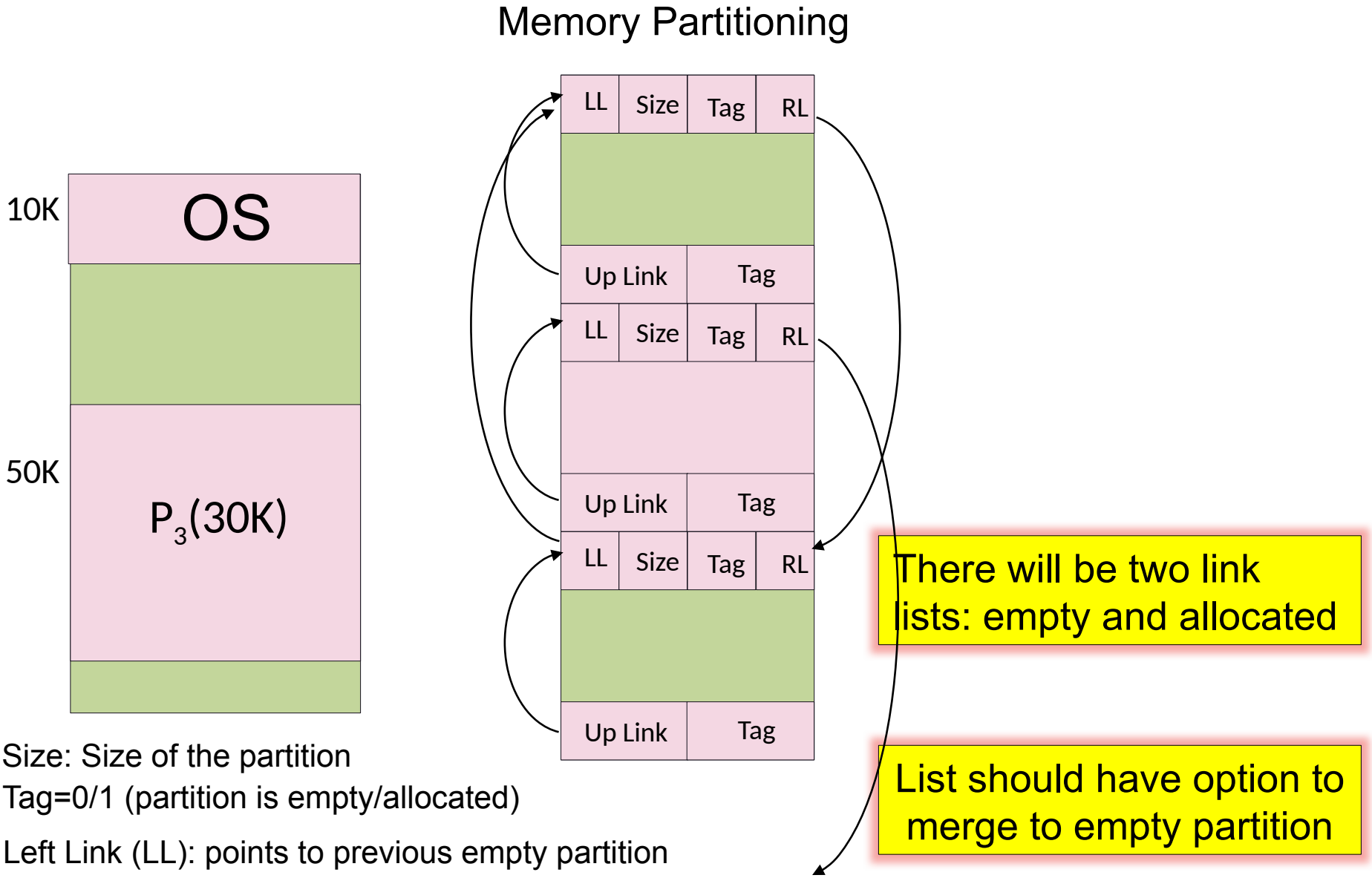
Right Link (RL): points to next empty partition

Up Link : points to its own LL

Fixed Memory Partitioning



Fixed Memory Partitioning



Size: Size of the partition

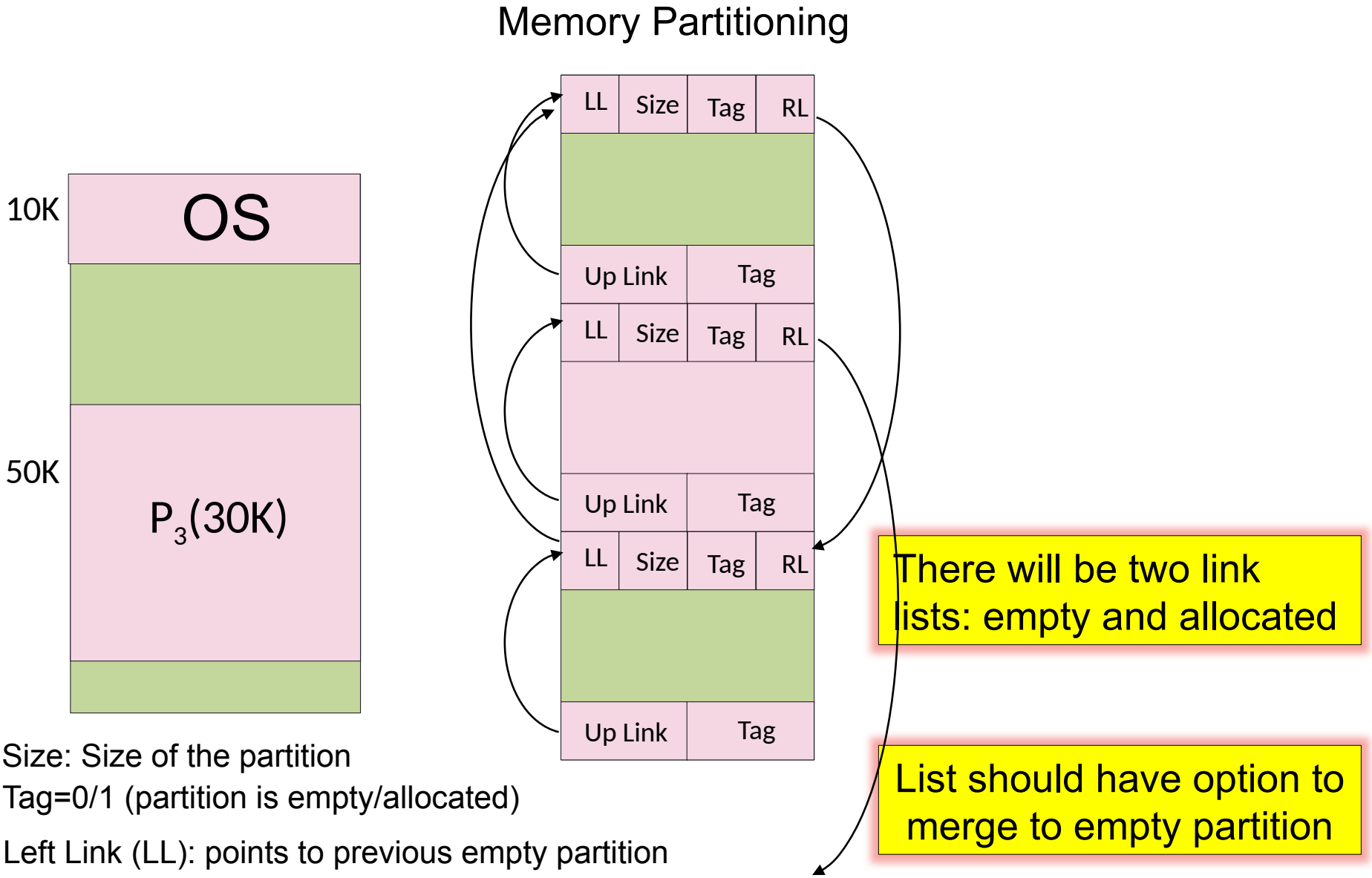
Tag=0/1 (partition is empty/allocated)

Left Link (LL): points to previous empty partition

Right Link (RL): points to next empty partition

Up Link : points to its own LL

Fixed Memory Partitioning



Size: Size of the partition

Tag=0/1 (partition is empty/allocated)

Left Link (LL): points to previous empty partition

Right Link (RL): points to next empty partition

Up Link : points to its own LL

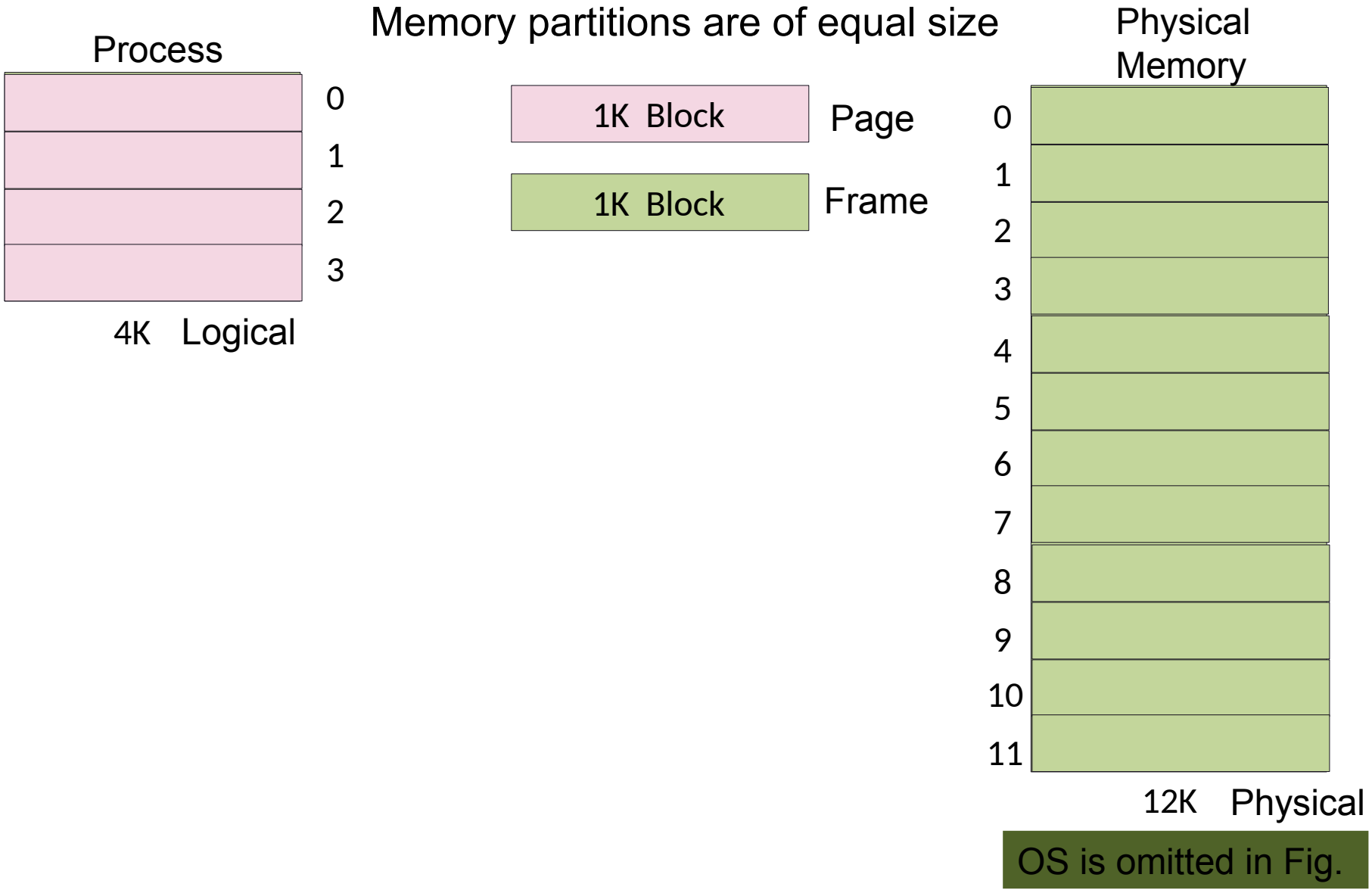
- Paged Memory Organization

Paging



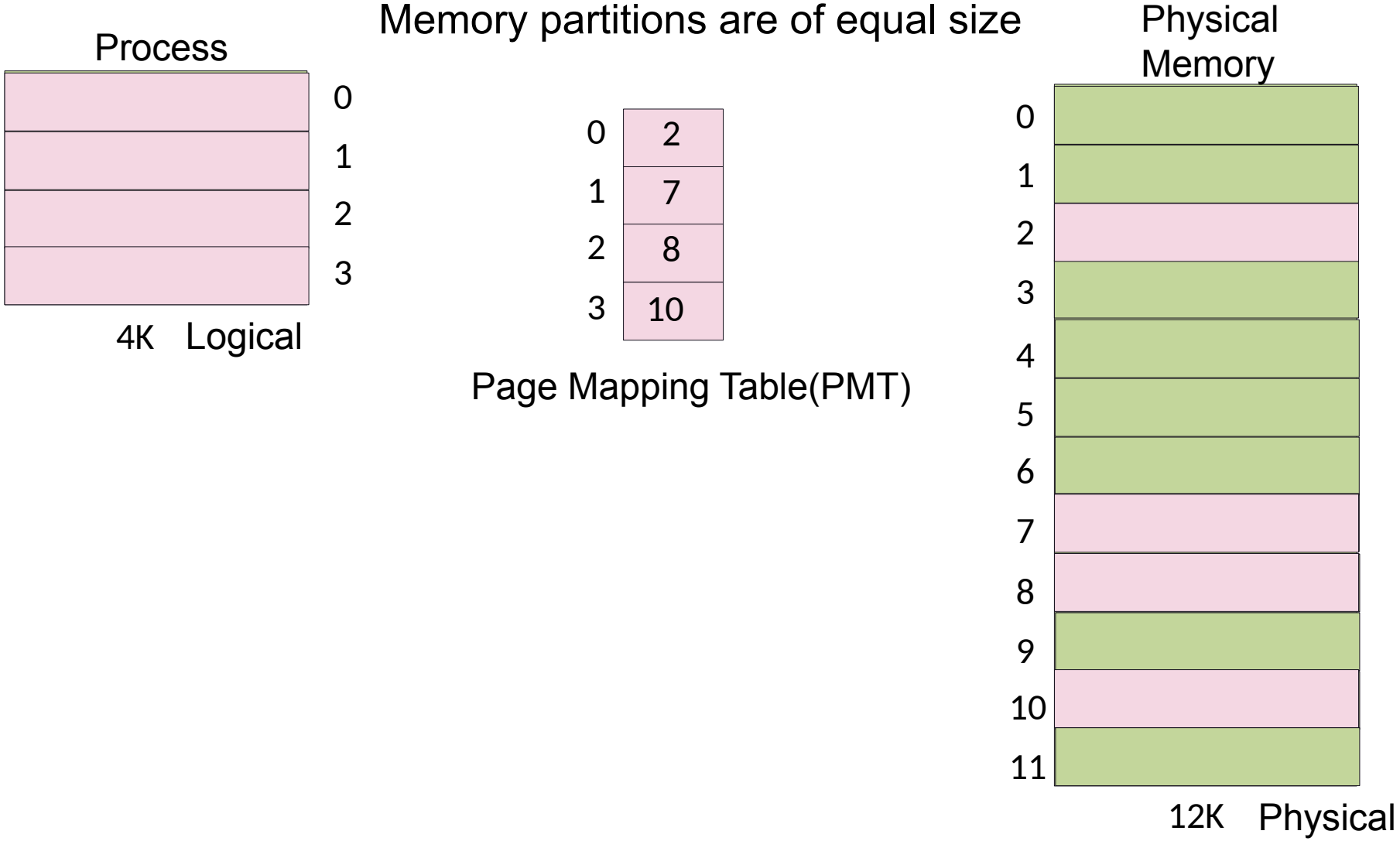
Paging

Memory partitions are of equal size



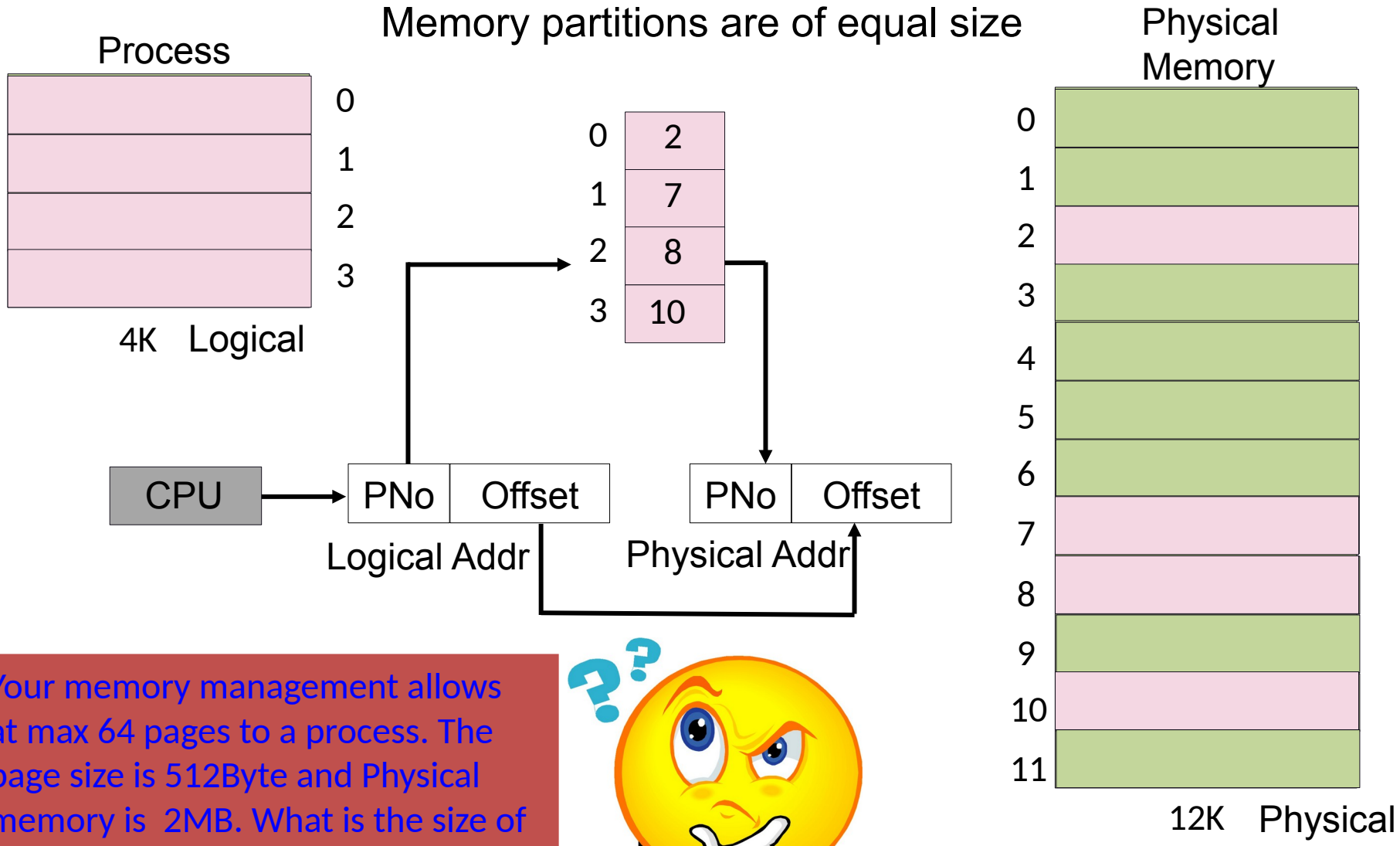
Paging

Memory partitions are of equal size



Paging

Memory partitions are of equal size



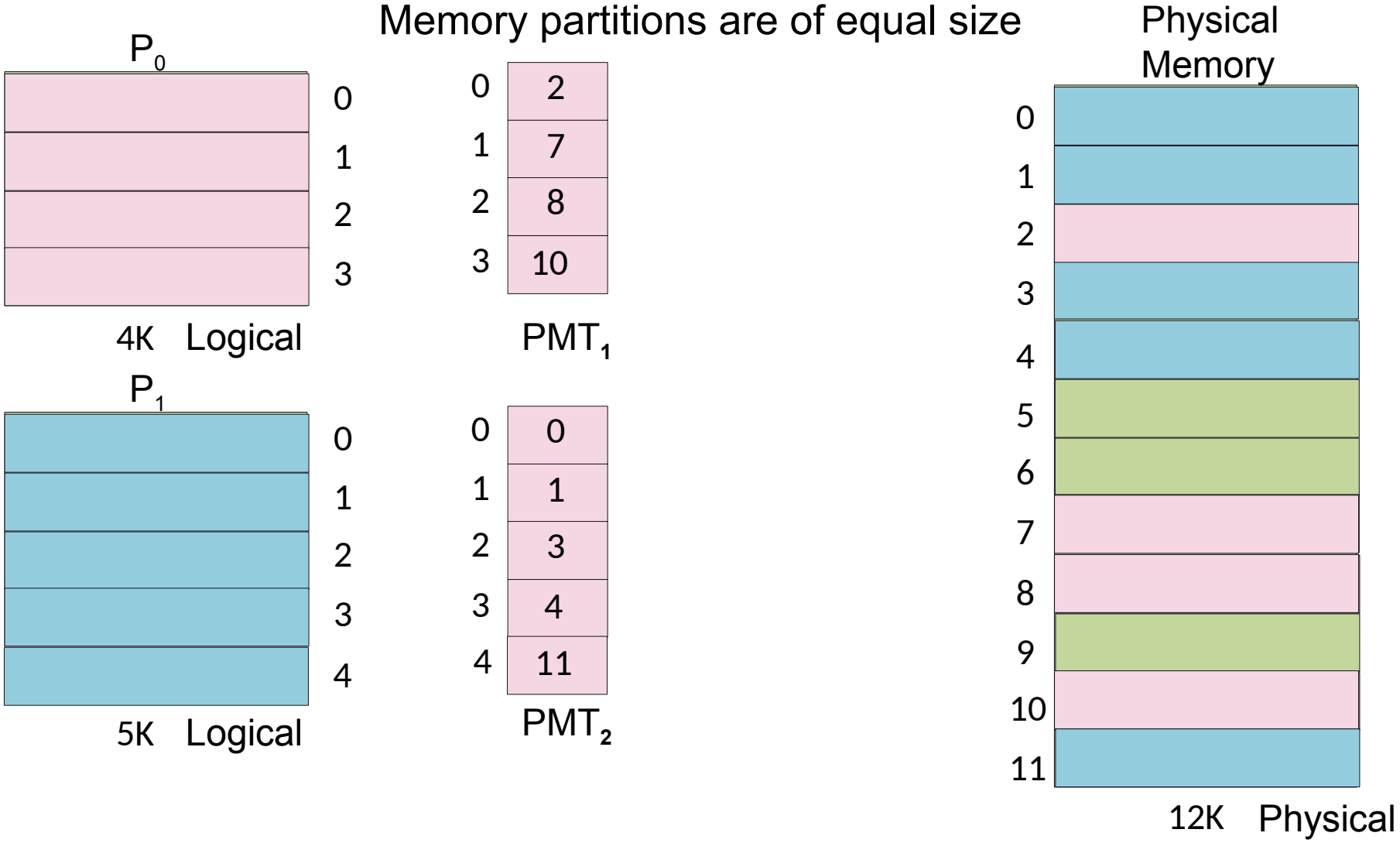
Your memory management allows at max 64 pages to a process. The page size is 512Byte and Physical memory is 2MB. What is the size of physical and logical address?



How to choose page/frame size?

Multiple Process in Paging

Memory partitions are of equal size

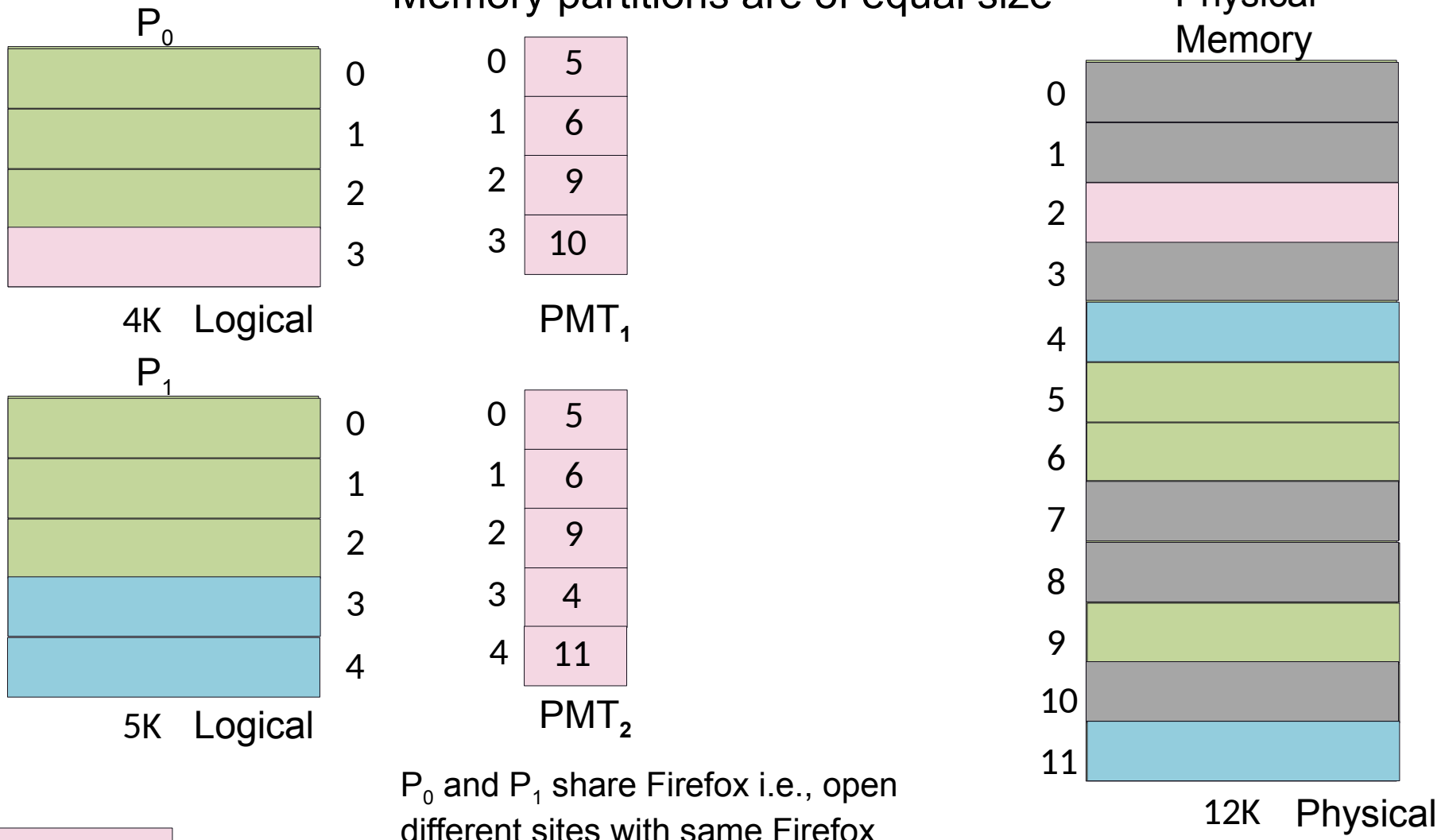


Pros: No internal fragmentation

Cons: Complex Mapping and external fragmentation

Code Sharing in Paging

Memory partitions are of equal size



P_0 and P_1 share Firefox i.e., open different sites with same Firefox

P0 Data

P1 Data

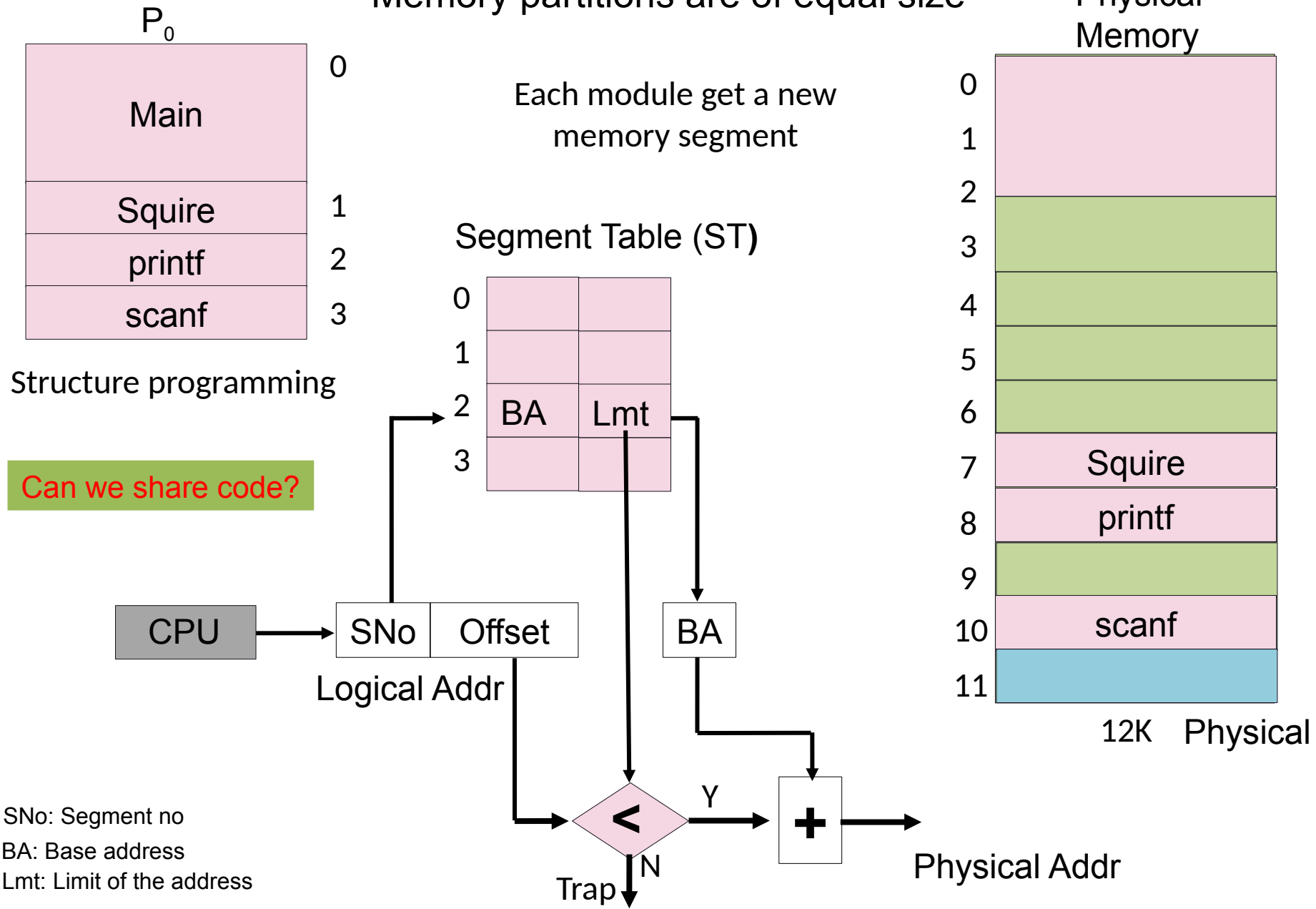
Code of Firefox

Firefox code should not be modified by P_0 and P_1
Reentrant code

- Segment Memory Organization

Paging

Memory partitions are of equal size



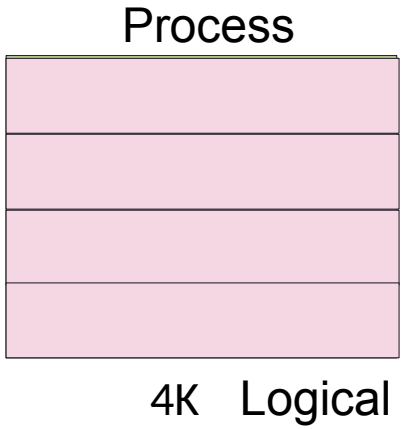
SNo: Segment no

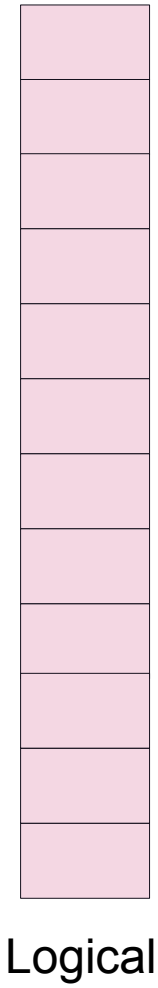
BA: Base address

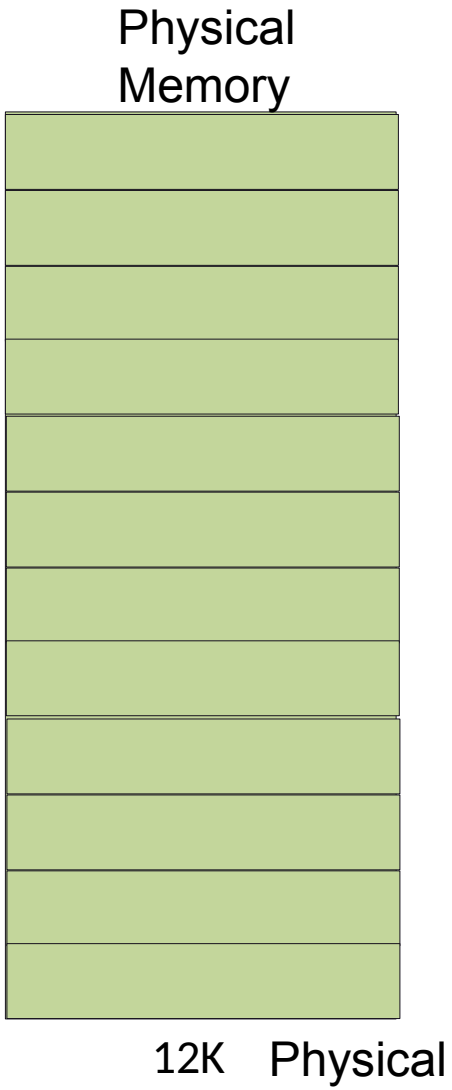
Lmt: Limit of the address

Paging

Memory partitions are of equal size

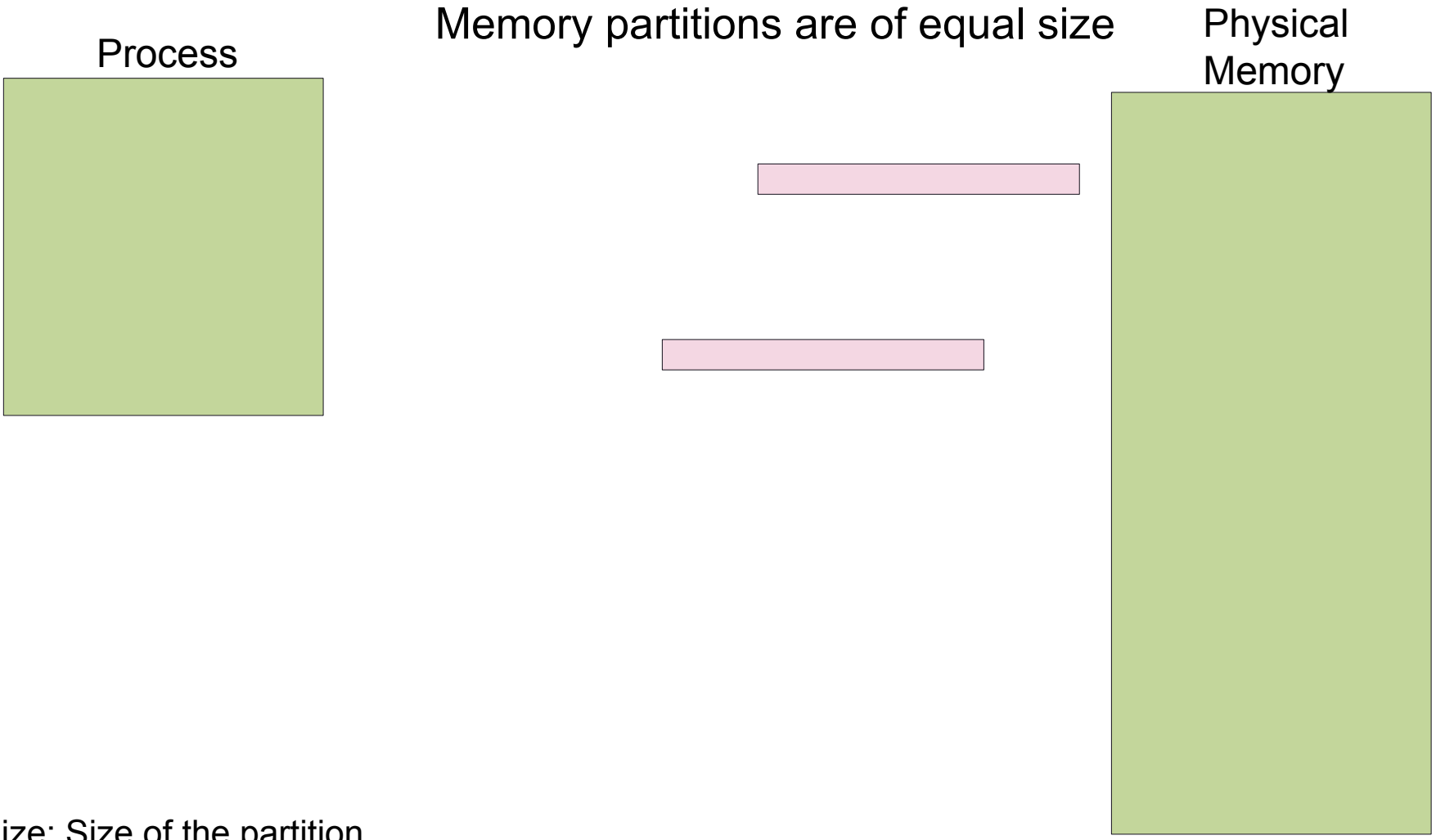






Paging

Memory partitions are of equal size



Size: Size of the partition

Tag=0/1 (partition is empty/allocated)

Left Link (LL): points to previous empty partition

Right Link (RL): points to next empty partition

Up Link : points to its own LL